

**PENGEMBANGAN ARSITEKTUR IOT TERDISTRIBUSI
BERBASIS MICROSERVICES DAN FIRMWARE MODULAR
PADA SISTEM OTOMASI BUDIDAYA AEROPONIK**

TUGAS AKHIR



Oleh

Alif Muhammad Rizky

NIM: 13322006

**PROGRAM STUDI TEKNIK FISIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI BANDUNG
2026**

ABSTRAK

PENGEMBANGAN ARSITEKTUR IOT TERDISTRIBUSI BERBASIS MICROSERVICES DAN FIRMWARE MODULAR PADA SISTEM OTOMASI BUDIDAYA AEROPONIK

Oleh

Alif Muhammad Rizky NIM: 13322006

(Program Studi Teknik Fisika)

Pertanian presisi pada budidaya aeroponik menuntut pemantauan dan kontrol mikroklimat secara kontinu. Namun, arsitektur monolitik konvensional memiliki keterikatan erat antarkomponen, rentan *single point of failure*, serta sulit diskalakan. Penelitian ini merancang, mengimplementasikan, dan mengevaluasi sistem otomasi aeroponik terdistribusi berbasis arsitektur *microservice* modular di server dan *firmware* modular pada *edge node* ESP32. Metode penelitian menggunakan kerangka *Design Science Research* (DSR). Pada sisi *edge*, ESP32 berbasis FreeRTOS *dual-core* dilengkapi antarmuka *Captive Web Portal* untuk konfigurasi pin, WiFi, serta pendaftaran input sensor dan output aktuator secara interaktif tanpa kompilasi ulang. ESP32 dirancang sebagai *node* MQTT modular berarsitektur *factory pattern* dan *protocol registry*. Penekanan utama modularitas terletak pada fleksibilitas input sensor (pembacaan otomatis berbagai protokol) dan output aktuator: pengguna atau *service* backend cukup mempublikasikan muatan *payload* JSON spesifik ke topik MQTT standar, lalu *firmware* ESP32 secara otomatis memetakan, mengurai, dan meneruskan perintah langsung ke output aktuator target yang sesuai. Pada sisi server, sistem menerapkan *database-per-service* pada empat layanan inti (Auth, Module, Control, Analytics) via NATS JetStream dan API Gateway Kong. Layanan kecerdasan buatan berbasis *Reinforcement Learning* (algoritma TD3) diintegrasikan untuk memberikan kontrol adaptif pada penjadwalan penyemprotan (*misting*), mengombinasikan data telemetri real-time dan ekstraksi fitur visual tanaman dari model pra-latih YOLOv8 via REST API secara *loose coupling*. Pengujian empiris memvalidasi penambahan *handler* I2C sensor INA219 via *Captive Portal* dan eksekusi perintah aktuator MQTT berbasis topik secara dinamis. Uji *chaos engineering* mengonfirmasi isolasi kegagalan antarlayanan penuh, sedangkan uji beban mencatat latensi *end-to-end* di bawah 2 detik pada persentil ke-95. Penelitian ini berkontribusi menyediakan arsitektur referensi IoT terdistribusi yang tangguh, adaptif, dan sangat modular.

Kata kunci: pertanian presisi, aeroponik, microservice, firmware modular, ESP32, MQTT, captive portal, reinforcement learning.

ABSTRACT

DEVELOPMENT OF A DISTRIBUTED IOT ARCHITECTURE BASED ON MICROSERVICES AND MODULAR FIRMWARE FOR AN AEROPONIC CULTIVATION AUTOMATION SYSTEM

By

Alif Muhammad Rizky

NIM: 13322006

(Engineering Physics Program)

Precision agriculture in aeroponic cultivation requires continuous microclimate monitoring and control. However, conventional monolithic architectures suffer from tight component coupling, vulnerability to single points of failure, and scalability limitations. This research designs, implements, and evaluates a distributed aeroponic automation system based on a modular microservice architecture at the server level and modular firmware on ESP32 edge nodes. The study employs the Design Science Research (DSR) framework. At the edge, the dual-core FreeRTOS-based ESP32 features a Captive Web Portal interface for interactive configuration of pins and WiFi, as well as the registration of sensor inputs and actuator outputs without requiring recompilation. The ESP32 is designed as a modular MQTT node utilizing factory pattern and protocol registry architectures. The modularity emphasizes flexibility in sensor inputs (supporting automatic reading across various protocols) and actuator outputs; users or backend services simply publish specific JSON payloads to standard MQTT topics, and the ESP32 firmware automatically maps, parses, and forwards commands directly to the appropriate target actuator. On the server side, the system implements a database-per-service approach across four core services (Auth, Module, Control, Analytics) using NATS JetStream and the Kong API Gateway. A Reinforcement Learning-based AI service (utilizing the TD3 algorithm) is integrated to provide adaptive control over misting schedules, combining real-time telemetry data with plant visual feature extraction from a pre-trained YOLOv8 model via a loosely coupled REST API. Empirical testing validated the dynamic addition of INA219 I2C sensor handlers via the Captive Portal and the execution of topic-based MQTT actuator commands. Chaos engineering tests confirmed complete inter-service failure isolation, while load tests recorded an end-to-end latency of under 2 seconds at the 95th percentile. This research contributes a robust, adaptive, and highly modular distributed IoT reference architecture.

Keywords: *precision agriculture, aeroponics, microservices, modular firmware, ESP32, MQTT, captive portal, reinforcement learning.*

**PENGEMBANGAN ARSITEKTUR IOT TERDISTRIBUSI
BERBASIS MICROSERVICES DAN FIRMWARE MODULAR
PADA SISTEM OTOMASI BUDIDAYA AEROPONIK
HALAMAN PENGESAHAN**

Oleh
Alif Muhammad Rizky NIM: 13322006
(Program Studi Teknik Fisika)

Institut Teknologi Bandung

Menyetujui
Tim Pembimbing

Tanggal 31 Agustus 2026

Pembimbing 1

Pembimbing 2

Ir. Estiyanti Ekawati, M.T., Ph.D
NIP. 196808052008012020

Faqihza Mukhlis, S.T., M.T., Ph.D
NIP. 121110001

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan penyusunan Tugas Akhir yang berjudul "Pengembangan Arsitektur IoT Terdistribusi Berbasis Microservices dan Firmware Modular pada Sistem Otomasi Budidaya Aeroponik". Penelitian ini diajukan sebagai salah satu syarat kelulusan Program Studi Sarjana Teknik Fisika, Fakultas Teknologi Industri, Institut Teknologi Bandung.

Penyelesaian Tugas Akhir ini tidak terlepas dari dukungan, bimbingan, serta kritik yang membangun dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis ingin menyampaikan ucapan terima kasih yang sebesar-besarnya kepada:

1. Ir. Estiyanti Ekawati, M.T., Ph.D selaku Dosen Pembimbing I yang telah memberikan bimbingan, arahan teknis yang tajam mengenai arsitektur sistem terdistribusi, serta diskusi evaluasi yang sangat mendalam selama proses penelitian.
2. Faqihza Mukhlis, S.T., M.T., Ph.D selaku Dosen Pembimbing II yang telah memberikan masukan krusial terkait integrasi perangkat keras, pengembangan firmware ESP32, dan optimasi infrastruktur perangkat lunak.
3. Keluarga penulis yang sudah menyemangati, mendoakan, dan mendukung penulis dari awal studi hingga menuntaskan tugas akhir ini.
4. Alsa, Abbiyu, dan rekan-rekan laboratorium PTIO yang telah membantu dan menemani penulis dalam menuntaskan studi dan tugas akhir di ITB.
5. Seluruh jajaran dosen dan staf tenaga kependidikan Program Studi Teknik Fisika ITB yang telah memfasilitasi dan mendidik penulis selama masa perkuliahan.
6. Serta teman-teman Teknik Fisika ITB angkatan 2022 dan HMFT ITB yang telah menemani dan mengisi kenangan penulis selama berkuliah di ITB hingga dapat menuntaskan tugas akhir.

Penulis menyadari bahwa Tugas Akhir ini masih memiliki berbagai keterbatasan dan ruang untuk pengembangan lebih lanjut, khususnya pada aspek optimasi algoritma dan perluasan cakupan pengujian. Oleh karena itu, penulis sangat terbuka terhadap setiap saran dan kritik yang konstruktif guna penyempurnaan di masa mendatang. Akhir kata, penulis berharap penelitian ini dapat memberikan kontribusi yang riil dan bermanfaat bagi kemajuan ilmu pengetahuan, khususnya di bidang rekayasa sistem IoT, komputasi terdistribusi, dan otomasi pertanian presisi.

Bandung, 31 Agustus 2026

Penulis

Alif Muhammad Rizky

1332200

DAFTAR ISI

ABSTRAK	i
HALAMAN PENGESAHAN.....	iii
KATA PENGANTAR.....	iv
DAFTAR ISI.....	vi
DAFTAR GAMBAR.....	ix
DAFTAR TABEL	xi
DAFTAR SINGKATAN DAN LAMBANG	xii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian	3
1.4 Lingkup Permasalahan, Asumsi, dan Hipotesis	4
1.5 Pendekatan, Metode, dan Pelaksanaan Penelitian.....	6
1.6 Sistematika Penulisan.....	8
BAB II DASAR TEORI.....	9
2.1 Sistem Aeroponik dan Pertanian Presisi	9
2.2 Arsitektur Komputasi Terdistribusi dan Microservices	9
2.3 Protokol Komunikasi dan Arsitektur <i>Event-Driven</i>	10
2.4 <i>Firmware</i> Modular dan Pemrograman Berorientasi Objek pada <i>Edge</i> 10	
2.5 Intelegensi Buatan untuk Kontrol Adaptif	12
2.6 Pengujian Kinerja dan Keandalan Sistem Terdistribusi.....	13
BAB III PERANCANGAN SISTEM DAN METODOLOGI	14
3.1 Arsitektur Sistem.....	14
3.2 Arsitektur Field Zone dan Station Zone.....	15
3.3 Perancangan Firmware Modular ESP32	16
3.3.1 FreeRTOS.....	16
3.3.2 Modular I/O : Factory Pattern & Registry.....	17
3.3.3 Vector Registry.....	19
3.3.4 Penambahan Protokol Baru I2C	20

3.3.5	Map Registry	20
3.4	Antarmuka Firmware: Captive Web Portal.....	21
3.5	Standarisasi Komunikasi MQTT.....	22
3.5.1	Parameter dan Kontrak Topik	22
3.5.2	Standar Output Telemetry dan Kendali Aktuator.....	22
3.6	Arsitektur Sistem Server	24
3.7	Ingesti Data Telemetry Module Service	25
3.8	Pemrosesan Data Analytics Service.....	27
3.9	Manajemen Kontrol Pada Control Service	28
3.10	Penambahan Layanan Pengolahan Citra	31
3.11	Algoritma Misting Adaptif dan Layanan Kontrol Adaptif.....	33
3.11.1	Algoritma Misting Adaptif.....	33
3.11.2	Layanan Kontrol Adaptif.....	35
3.12	Dashboard.....	38
3.13	Skenario Pengujian.....	39
BAB IV	HASIL PENELITIAN.....	40
4.1	Pengujian Modularitas Sensor, Aktuator & Konfigurasi Dinamis via Antarmuka Web Captive Portal	40
4.1.1	Skenario Penambahan Sensor dan Aktuator	40
4.2	Pengujian Alur Integrasi End-to-End via Antarmuka Dashboard.....	44
4.2.1	Registrasi Module, Node, dan Tag Mapping via Dashboard .	44
4.2.2	Verifikasi Tampilan Dashboard, Analytics dan Pengujian Control.....	45
4.3	Hasil Pengujian Algoritma RL TD3	47
4.4	Evaluasi Tahap I: Konfigurasi Minimal tanpa Service Tambahan	49
4.4.1	Pengujian Unit Test Fitur Layanan	49
4.4.2	Pengujian Stress Test.....	49
4.4.3	Pengujian Chaos Engineering	53
4.5	Evaluasi Tahap II: Konfigurasi Lengkap dengan Service Tambahan	54
4.5.1	Integrasi Layanan Tambahan ML, model-control, model- controller	54
4.5.2	Pengujian Unit Test Fitur Layanan	55

4.5.3	Pengujian Stress Test.....	56
4.5.4	Pengujian Chaos Engineering Test.....	59
4.6	Analisis Komparatif dan Pembahasan.....	60
4.6.1	Analisis Kinerja Troughput Latensi dan Resource.....	60
4.6.2	Pengaruh Penambahan Node.....	61
4.6.3	Pengaruh Penambahan Layanan AI/ML	62
4.6.4	Keandalan Sistem Menangani Service yang Gagal.....	62
BAB V KESIMPULAN.....		64
4.7	Kesimpulan	64
4.8	Saran.....	65
DAFTAR PUSTAKA		66
LAMPIRAN.....		67

DAFTAR GAMBAR

Gambar 3.1 General System Architecture Diagram	14
Gambar 3.2 Sensor and Actuator Configuration.....	15
Gambar 3.3 Activity Diagram - Core Runtime ESP32 FreeRTOS.....	16
Gambar 3.4 Class Diagram - Factory Pattern & Registry ProtocolHandler	18
Gambar 3.5 Captive Web Portal Interface	21
Gambar 3.6 Sequence Diagram — Telemetry Flow	23
Gambar 3.7 Sequence Diagram — Actuator Command Flow.....	23
Gambar 3.8 Microservice Architecture Diagram.....	24
Gambar 3.9 Data Flow Diagram — Telemetry Pipeline.....	25
Gambar 3.10 Data Flow Diagram — Analytics Pipeline.....	27
Gambar 3.11 Sequence Diagram — Control Service Command.....	29
Gambar 3.12 Class Diagram — Model Registry	31
Gambar 3.13 Sequence Diagram — AI Detect Inference Flow.....	32
Gambar 3.14 Sequence Diagram — Adaptive Control Flow	36
Gambar 3.15 Dashboard.....	38
Gambar 4.1 Konfigurasi Input GPIO	41
Gambar 4.2 Konfigurasi Output GPIO	41
Gambar 4.3 Konfigurasi Sensor Modbus.....	42
Gambar 4.4 Wiring Diagram Sensor INA219.....	43
Gambar 4.5 Konfigurasi Sensor I2C	43
Gambar 4.6 Tampilan Halaman Monitor	45
Gambar 4.7 Grafik Historis Telemetri	45
Gambar 4.8 Hasil Pengolahan Data Halaman Analytics	46
Gambar 4.9 Tampilan Halaman Control.....	46
Gambar 4.10 Tampilan Command Log Konfirmasi Aktuator	47
Gambar 4.11 30-Day Simulation Linked Overview (Growth, Control, Weather, Response)	48
Gambar 4.12 Overall Unit Test Distribution Phase 1	49
Gambar 4.13 Throughput & Error vs Concurrency Phase 1	50
Gambar 4.14 P95 Latency & Error vs Concurrency Phase 1.....	50
Gambar 4.15 MQTT Pub-Sub Latency P95 Phase 1	51
Gambar 4.16 Publish Troughput MQTT Phase 1	51
Gambar 4.17 Ingest Throughput (in_msg/s NATS) Phase 1	52
Gambar 4.18 Transaction in TimescaleDB (tps) Phase 1	52
Gambar 4.19 Resource Usage Summary Phase 1	52
Gambar 4.20 Core Service Isolation Matrix Phase 1	54
Gambar 4.21 Kondisi Penjadwalan Waktu Siang.....	55
Gambar 4.22 Kondisi Penjadwalan Waktu Malam.....	55
Gambar 4.23 Overall Test Distribution Fase 2	56
Gambar 4.24 Throughput & Error vs Concurrency Phase 2.....	56
Gambar 4.25 Latency vs Concurrency Phase 2	57
Gambar 4.26 MQTT Pub-Sub Latency P95 Phase 2	57
Gambar 4.27 Publish Troughput MQTT Phase 2	58

Gambar 4.28 Ingest Throughput (in_msg/s NATS) Phase 2	58
Gambar 4.29 Transaction in TimescaleDB (tps) Phase 2	58
Gambar 4.30 Resource Usage Summary Phase 2	59
Gambar 4.31 Core Service Isolation Matrix Phase 2	59

DAFTAR TABEL

Tabel 3.1 Ruang Keadaan (State Space) - 10D	33
Tabel 3.2 Ruang Aksi (Action Space) - 3D	34
Tabel 4.1 Ringkasan Kinerja Troughput Latensi dan Resource	60

DAFTAR SINGKATAN DAN LAMBANG

SINGKATAN	Nama	Pemakaian pertama kali pada halaman
IoT	<i>Internet of Think</i>	6
MQTT	<i>Message Queuing Telemetry Transport</i>	1
TD3	<i>Twin Delayed Deep Deterministic Policy Gradient</i>	1
YOLO	<i>You Only Look Once</i>	2
EC	<i>Electrical Conductivity</i>	4
RBAC	<i>Role-Based Access Control</i>	38
JWT	<i>JSON Web Token</i>	67
REST	<i>Representational State Transfer</i>	4
API	<i>Application Programming Interface</i>	1
JSON	<i>JavaScript Object Notation</i>	16
I2C	<i>Inter-Integrated Circuit</i>	2
WS	<i>WebSocket</i>	4
CPU	<i>Central Processing Unit</i>	4
ML	<i>Machine Learning</i>	3
RL	<i>Reinforcement Learning</i>	9
RTOS	<i>Real-Time Operating System</i>	1

BAB I

PENDAHULUAN

1.1 Latar Belakang

Budidaya aeroponik sangat sensitif terhadap perubahan kondisi lingkungan, sehingga memerlukan kontrol *misting* yang presisi berbasis parameter mikroklimat seperti suhu udara, kelembapan, *electrical conductivity* (EC), dan pH larutan nutrisi. Oleh karena itu, dibutuhkan sebuah sistem terintegrasi yang berfungsi melakukan akuisisi data sensor secara kontinu serta mengendalikan aktuator secara responsif dan andal [1].

Namun, mayoritas sistem IoT konvensional saat ini masih berbasis arsitektur monolitik yang memiliki keterikatan erat antarkomponen (*tight coupling*). Hal ini menyebabkan sistem sulit diskalakan dan tidak fleksibel saat terjadi ekspansi, seperti penambahan sensor atau aktuator baru, karena menuntut modifikasi logika utama program serta kompilasi ulang *firmware*. Oleh karena itu, diperlukan arsitektur sistem yang modular untuk mengakomodasi perkembangan perangkat keras secara dinamis tanpa perlu membangun atau mengompilasi ulang kode program dari awal [2].

Oleh karena itu, penelitian ini merancang pendekatan modularitas Guna mengatasi keterbatasan tersebut, penelitian ini merancang pendekatan modularitas berbasis *dual-layer distribution* pada sisi *edge* dan *server*. Pada sisi *edge*, *firmware* ESP32 dirancang menggunakan *factory pattern* dan *protocol registry* untuk mendukung konfigurasi perangkat lunak secara dinamis [3]. Sementara itu pada sisi *server*, sistem menerapkan arsitektur *microservice* dengan pola database pada setiap service (*database-per-service*). Dengan metode ini, setiap *service* dapat terhubung secara terisolasi dan fleksibel (*loose coupling*), sehingga dapat dikembangkan, diperbarui, dan dikelola secara independen tanpa menimbulkan efek domino berupa kegagalan sistem secara menyeluruh (*cascading failure*) [4].

Selain modularitas arsitektur, sistem pemantauan dan kontrol pada budidaya aeroponik juga memerlukan mekanisme kendali yang presisi dan adaptif. Untuk memenuhi kebutuhan tersebut, dikembangkan layanan kontrol adaptif berbasis

algoritma *Reinforcement Learning* dengan metode *Twin Delayed Deep Deterministic Policy Gradient* (TD3) [5]. Layanan ini mengombinasikan data telemetri iklim mikro secara *real-time* dengan ekstraksi fitur visual kondisi tanaman dari model visi komputer YOLOv8 secara *loose coupling*. Pendekatan ini memungkinkan penyesuaian jadwal penyemprotan (*misting*) dan pengoperasian katup secara fleksibel tanpa menimbulkan keterikatan ketat (*tight coupling*) dengan layanan inti sistem.

Namun, seiring bertambahnya kompleksitas sistem dan ekspansi jumlah *node*, timbul *trade-off* berupa potensi lonjakan latensi, *throughput*, serta utilisasi sumber daya. Oleh karena itu, pengujian empiris dilakukan melalui pengujian beban (*stress test*) dan uji ketahanan (*chaos engineering*) [6]. Pengujian ini bertujuan untuk membuktikan bahwa arsitektur sistem terdistribusi yang dirancang mampu mempertahankan stabilitas kinerja, menjaga latensi *end-to-end* tetap rendah, serta mengisolasi kegagalan penuh antar-layanan (*zero cascade failure*) saat beroperasi.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam penelitian ini adalah:

1. Bagaimana merancang dan mengimplementasikan arsitektur sistem pemantauan dan kontrol aeroponik terdistribusi yang mampu memisahkan dependensi komputasi antara sisi *server* (*microservices*) dan sisi *edge* (*firmware* modular) guna mengatasi keterbatasan arsitektur monolitik?
2. Bagaimana mengimplementasikan prinsip pemrograman berorientasi objek melalui penerapan pola *Factory Pattern* dan *Protocol Registry* pada *firmware* ESP32 agar penambahan maupun konfigurasi protokol sensor/aktuator baru (seperti I2C sensor INA219) dapat dilakukan secara dinamis melalui antarmuka *Captive Web Portal* tanpa proses kompilasi ulang (*recompilation*)?
3. Bagaimana merancang dan mengimplementasikan mekanisme ingesti data telemetri real-time serta kontrol terdistribusi berbasis arsitektur

microservice dengan menerapkan prinsip *database-per-service*, komunikasi berbasis event via MQTT dan NATS JetStream, serta API Gateway Kong?

4. Bagaimana mengintegrasikan layanan kecerdasan buatan berbasis deteksi visual YOLOv8 dan *Reinforcement Learning* TD3 secara *loose coupling* ke dalam arsitektur *microservice* untuk menghasilkan kontrol penjadwalan penyemprotan (*misting*) yang adaptif berdasarkan kondisi fisik dan iklim mikro tanaman?
5. Sejauh mana keandalan isolasi kegagalan (*chaos engineering*) dan kinerja sistem (latensi end-to-end, *throughput*, serta utilisasi sumber daya) saat diuji dalam kondisi konfigurasi minimal maupun konfigurasi beban penuh (*stress test*)?

1.3 Tujuan Penelitian

Penelitian ini bertujuan untuk:

1. Merancang dan menguji arsitektur sistem pemantauan dan kontrol aeroponik terdistribusi melalui penerapan pendekatan modularitas dua lapis (*dual-layer modularity*) pada sisi *server* dan *edge*.
2. Mengimplementasikan firmware ESP32 berbasis FreeRTOS dual-core dengan pola *Factory Pattern* dan *Protocol Registry* untuk mendukung konfigurasi sensor/aktuator secara dinamis.
3. Membuktikan modularitas firmware secara empiris melalui penambahan dan konfigurasi handler protokol I2C (sensor daya INA219) via antarmuka Captive Web Portal tanpa rekompilasi kode.
4. Membangun dan menguji mekanisme ingesti data telemetri real-time serta alur kontrol terdistribusi yang terintegrasi pada arsitektur *microservice* dengan menerapkan prinsip *database-per-service*, bus komunikasi event-driven (MQTT dan NATS JetStream), serta pengelolaan pintu masuk API menggunakan Kong API Gateway.

5. Mengintegrasikan model *Reinforcement Learning* TD3 untuk penyesuaian jadwal penyemprotan adaptif dan pengolahan citra YOLOv8 via REST API/ML Service secara non-blocking (*loose coupling*).
6. Mengevaluasi kinerja kuantitatif sistem melalui pengujian unit fitur (*unit test*), uji beban (*stress test*) untuk mengukur latensi dan *throughput*, serta uji ketahanan (*chaos engineering*) untuk memastikan isolasi kegagalan penuh antar-layanan (*zero cascade failure*).

1.4 Lingkup Permasalahan, Asumsi, dan Hipotesis

Agar cakupan penelitian tetap terarah dan terukur, batasan ruang lingkup masalah ditetapkan sebagai berikut:

1. Permasalahan dibatasi pada perancangan, implementasi, dan evaluasi *dual-layer modularity*, yaitu arsitektur *microservice* pada sisi server dan *modular configuration-driven firmware* berbasis FreeRTOS dual-core pada mikrokontroler ESP32.
2. Pengujian modularitas *edge* dibatasi pada pembuktian integrasi handler protokol I2C untuk sensor daya (INA219) via antarmuka *Captive Web Portal* (config.json) tanpa perlu melakukan kompilasi ulang kode (*recompilation*).
3. Batasan lingkup penelitian difokuskan pada pengujian pipa ingesti data telemetri (*sistem existing*) dan pengintegrasian layanan kontrol adaptif (*sistem tambahan*). Pipa ingesti menguji alur data dari *Module Service*, NATS JetStream, hingga TimescaleDB (*Analytics Service*) dengan skema *database-per-service*. Layanan tambahan mencakup pengembangan *model-control* dan *model-controller* yang terhubung ke sistem inti via Kong API Gateway untuk eksekusi kendali otonom. Layanan pendukung *existing* lainnya (*Auth, Alert, Notification, Stream, Audit, Export, WS-Gateway*) diuji sebatas penopang operasional.

4. Penggunaan modul AI/ML pada sistem tambahan dibatasi pada alur integrasi *loose coupling* via REST API menggunakan model YOLOv8 (ekstraksi visual akar/umbi pada *ML Service*) dan *Reinforcement Learning* TD3 (keputusan penyemprotan adaptif). Evaluasi AI/ML berfokus pada efektivitas integrasi data dan dampaknya terhadap kinerja arsitektur terdistribusi (latensi, *throughput*, sumber daya, dan *chaos engineering*), bukan pada optimasi atau pengembangan algoritma AI/ML itu sendiri.
5. Sensor yang diintegrasikan dibatasi pada parameter suhu udara, kelembapan udara, suhu larutan nutrisi, *Electrical Conductivity* (EC), pH larutan, serta parameter daya I2C (INA219). Aktuator dibatasi pada pompa *misting* dan katup (*valve*).
6. Evaluasi kinerja kuantitatif dibatasi pada pengukuran latensi end-to-end (target P95 < 2 detik), *throughput* (RPS & pesan/detik), utilisasi sumber daya (CPU & RAM), serta ketahanan sistem melalui uji isolasi kegagalan (*chaos engineering matrix*).

Dalam pelaksanaan penelitian ini, digunakan asumsi-asumsi dasar sebagai berikut:

1. Jaringan WiFi lokal antara ESP32 dan server diasumsikan memiliki kualitas yang memadai untuk transmisi data telemetri dengan interval 1 detik secara stabil (latensi <500 ms, *packet loss* <1%).
2. Server pengujian diasumsikan memiliki spesifikasi komputasi yang memadai untuk menjalankan seluruh tumpukan kontainer Docker (*Docker Compose*) secara bersamaan.
3. Seluruh sensor fisik (suhu, kelembapan, EC, pH, INA219) diasumsikan telah dikalibrasi dan memberikan pembacaan parameter fisik secara valid.
4. Model YOLOv8 pra-latih dan agen TD3 yang dirancang diasumsikan siap menjalankan tugas inferensi untuk memberikan input parameter aksi kontrol secara konsisten selama skenario pengujian berlangsung.

5. Seluruh *microservice* diasumsikan dapat saling berkomunikasi melalui jaringan internal Docker Compose tanpa adanya hambatan *firewall* internal.
6. Seluruh tumpukan arsitektur dan layanan *server backend* diasumsikan telah selesai dikembangkan dan berfungsi penuh, sehingga fokus pembahasan dan evaluasi laporan ini berpusat pada penjelasannya mengenai modularitas sistem, mulai dari pipa ingesti data telemetri hingga manajemen eksekusi kontrol aktuator.

Hipotesis awal yang diajukan dalam penelitian ini adalah:

1. Penerapan pola *Factory Pattern* dan *Protocol Registry* pada firmware ESP32 berbasis FreeRTOS memungkinkan penambahan handler protokol komunikasi baru (seperti I2C INA219) secara dinamis tanpa mengubah kode logika inti (*main loop*) dan tanpa proses kompilasi ulang proyek.
2. Arsitektur *microservice* dengan prinsip *database-per-service* dan komunikasi *event-driven* via NATS JetStream pada layanan inti (Auth, Module, Control, Analytics) mampu mengisolasi kegagalan satu layanan sehingga layanan lainnya tetap beroperasi secara normal.
3. Sistem mampu melayani penambahan jumlah node sensor tanpa penurunan kinerja yang signifikan latensi end-to-end tetap <2 detik pada persentil ke-95.
4. Integrasi model TD3 yang dikembangkan mampu mengeksekusi penyesuaian jadwal penyemprotan secara adaptif berdasarkan kombinasi telemetri dan deteksi visual.

1.5 Pendekatan, Metode, dan Pelaksanaan Penelitian

Penelitian ini menggunakan pendekatan rekayasa sistem terapan (*applied engineering research*) dengan kerangka kerja *Design Science Research* (DSR). Pelaksanaan penelitian secara garis besar dibagi menjadi empat fase utama, yaitu:

1. **Fase Perancangan Sistem:** Merancang arsitektur *dual-layer modularity* yang memadukan firmware ESP32 berbasis FreeRTOS (*factory pattern & protocol registry*) di sisi *edge*, serta 13 *microservices* mandiri di sisi server. Sistem menerapkan prinsip *database-per-service* (MariaDB, TimescaleDB, Redis, MinIO) dengan perantara komunikasi NATS JetStream, MQTT, WebSocket dan Kong API Gateway.
2. **Fase Implementasi:** Membangun seluruh komponen secara bertahap menggunakan Docker Compose, meliputi infrastruktur jaringan/database, layanan inti (*Auth, Module, Analytics, Control*), layanan pendukung, modul AI (TD3 & YOLOv8), serta antarmuka (*Captive Web Portal* dan *React Dashboard*).
3. **Fase Pengujian dan Verifikasi:** Pengujian dilakukan secara bertahap dalam dua skala konfigurasi, yaitu Tahap I (Konfigurasi Minimal) dan Tahap II (Konfigurasi Penuh dengan AI/ML). Tahap ini mencakup uji fungsional (*unit test*) untuk memverifikasi seluruh fitur layanan melalui Kong Gateway (/v1), serta uji beban (*stress test*) guna mengukur *throughput* dan latensi P95 pada jalur API maupun ingesti telemetri di bawah beban 4 hingga 10 node ESP32. Ketahanan sistem diuji melalui *chaos engineering* dengan mematikan *microservice* dan database secara paksa untuk membuktikan isolasi kegagalan penuh (*zero cascade failure*). Selain itu, pengujian modularitas *edge* dilakukan dengan mengintegrasikan handler I2C untuk sensor daya INA219 via *Captive Web Portal* tanpa proses kompilasi ulang firmware.
4. **Fase Evaluasi dan Analisis:** Menganalisis metrik kuantitatif (latensi end-to-end P95 < 2 detik, *throughput*, utilisasi CPU/RAM, dan ketersediaan layanan) untuk memverifikasi hipotesis penelitian.

1.6 Sistematika Penulisan

Sistematika penyusunan laporan tugas akhir ini dibagi menjadi lima bab utama.

1. Bab ini menguraikan latar belakang permasalahan terkait pentingnya kendali presisi pada budidaya aeroponik, keterbatasan sistem IoT monolitik, serta solusi arsitektur terdistribusi. Selain itu, bab ini memuat perumusan masalah, tujuan penelitian, manfaat penelitian, batasan masalah dan asumsi, serta sistematika penulisan laporan.
2. Bab II memaparkan tinjauan pustaka yang mencakup teori dasar sistem aeroponik, arsitektur komputasi terdistribusi, protokol komunikasi dan arsitektur *event-driven*, firmware modular dan penerapan pemograman berorientasi objek, intelegensi buatan untuk kontrol adaptif, dan pengujian kinerja dan keandalan sistem terdistribusi.
3. Bab III menguraikan metode penelitian dan perancangan arsitektur terdistribusi *dual-layer modularity*, yang mencakup pembagian zona fungsional sistem, perancangan *firmware* ESP32 berbasis FreeRTOS dengan pola *Factory Pattern* dan *Protocol Registry*, antarmuka *Captive Web Portal*, standarisasi komunikasi MQTT, perancangan *microservices* inti berprinsip *database-per-service*, integrasi *loose-coupling* model AI (YOLOv8 dan TD3), perancangan antarmuka *Dashboard*, serta penetapan skenario pengujian kinerja dan keandalan sistem
4. Bab IV menguraikan hasil implementasi sistem, proses pengujian fungsional dan performa (*unit-test*, *stress-test*, *chaos-engineering*, pembuktian modularitas firmware via protokol I2C untuk sensor INA219, dan pengujian alur integrasi algoritma kontrol), analisis metrik kinerja (latensi, throughput, isolasi kegagalan), serta pembahasan perbandingan terhadap hipotesis yang diajukan.
5. Bab V berisi rincian kesimpulan akhir dari penelitian termasuk pembuktian empiris empat hipotesis keterbatasan sistem yang teridentifikasi dan rekomendasi untuk kajian dan pengembangan lanjutan.

BAB II

DASAR TEORI

2.1 Sistem Aeroponik dan Pertanian Presisi

Pertanian presisi mengandalkan data *real-time* sensor untuk kendali nutrisi dan aktuator secara otomatis. Pada aeroponik, akar tumbuh di udara dan disemprot nutrisi secara berkala sehingga suhu, kelembapan, *electrical conductivity* (EC), dan pH bersifat kritis serta wajib dipantau kontinu. Integrasi sensor cerdas dan kendali otomatis berpengaruh langsung terhadap stabilitas pertumbuhan tanaman, sehingga sistem harus mengakuisisi telemetri secara kontinu dan responsif [1].

2.2 Arsitektur Komputasi Terdistribusi dan Microservices

Arsitektur *microservices* mendefinisikan sebuah aplikasi sebagai kumpulan layanan kecil yang dapat di-*deploy* secara independen dan berkomunikasi melalui mekanisme yang ringan. Setiap layanan menerapkan prinsip *single responsibility* dan dipisahkan berdasarkan *bounded context*, sehingga perubahan pada satu layanan tidak memaksa penyebaran ulang (*redployment*) layanan lain. Pendekatan ini meminimalkan keterikatan ketat (*tight coupling*) dan menjadi landasan bagi pengembangan, pengujian, serta skala operasional yang terpisah [7].

Salah satu prinsip utama arsitektur *microservices* adalah *database-per-service*, yakni setiap layanan mengelola basis data secara mandiri tanpa adanya pemakaian basis data bersama (*shared database*) antar-layanan. Isolasi fisik dan logis ini mencegah terjadinya kegagalan berantai (*cascading failure*) serta meniadakan *Single Point of Failure* (SPOF) pada lapisan data, sehingga kerusakan pada satu basis data tidak meruntuhkan seluruh akses data sistem. Dalam praktiknya, pendekatan ini menerapkan pola *Eventually Consistent* untuk mengelola konsistensi data terdistribusi secara asinkron tanpa memblokir proses komputasi utama. Selain itu, pengelolaan antarmuka luar dikendalikan oleh *API Gateway* (seperti Kong) yang berfungsi sebagai titik masuk tunggal (*single point of entry*) untuk penanganan *routing*, autentikasi, serta manajemen evolusi API. Meskipun *API Gateway* berpotensi menjadi SPOF di tingkat arsitektur, risikonya dimitigasi

melalui penerapan *redundancy* dan pemantauan kondisi kesehatan (*health check*) secara berkala untuk menjamin ketersediaan sistem tetap tinggi [2].

2.3 Protokol Komunikasi dan Arsitektur *Event-Driven*

Komunikasi data pada sistem terdistribusi mengombinasikan protokol MQTT untuk transmisi data dari perangkat *edge* ke *server* serta NATS JetStream untuk pertukaran pesan berbasis *event* antar-layanan di dalam *server* [8], [9].

MQTT (*Message Queuing Telemetry Transport*) merupakan protokol komunikasi *publish-subscribe* berbasis *broker* yang dirancang sangat ringan, hemat *bandwidth*, dan efisien sumber daya. Karakteristik ini menjadikannya standar utama untuk transmisi data telemetri dari perangkat *edge* terpencil atau terbatas (*resource-constrained*). Protokol ini memisahkan pengirim (*publisher*) dan penerima (*subscriber*) secara independen, serta menyediakan opsi *Quality of Service* (QoS) untuk menjamin keandalan pengiriman data pada jaringan yang tidak stabil [8].

Sementara itu, NATS JetStream merupakan infrastruktur pesan berbasis *Event-Driven Architecture* (EDA) berkinerja tinggi yang mengelola komunikasi asinkron antar-*microservice* di sisi *server*. Berbeda dengan arsitektur *request-response* sinkron yang rentan terhadap pemblokiran (*blocking*), NATS JetStream menyediakan fitur *persistence*, pemutaran ulang pesan (*message replay*), dan *durable consumer*. Fitur-fitur dasar ini memungkinkan pertukaran data antar-layanan berlangsung secara *loose coupling*, *non-blocking*, serta tahan terhadap kehilangan pesan saat salah satu layanan mengalami kegagalan [9].

2.4 *Firmware* Modular dan Pemrograman Berorientasi Objek pada *Edge*

Firmware merupakan jenis perangkat lunak (*software*) tingkat rendah (*low-level*) yang ditanamkan secara langsung pada memori *non-volatile* mikrokontroler untuk menjembatani komunikasi antara komponen elektronik fisik (seperti sensor, aktuator, dan register prosesor) dengan logika sistem [10]. Pada sistem terbenam (*embedded system*), *firmware* bertanggung jawab mengelola eksekusi instruksi masukan/keluaran (I/O) serta memastikan perangkat dapat menjalankan tugas akuisisi data dan kontrol secara otonom. Penerapan arsitektur *firmware* modern

menuntut prinsip rekayasa perangkat lunak yang fleksibel guna mengatasi sifat kaku (*rigid*) pada pendekatan konvensional. Hal ini dicapai melalui integrasi konsep Pemrograman Berorientasi Objek (*Object-Oriented Programming / OOP*) dan pola desain (*design pattern*) pada perangkat keras tingkat *edge* [3].

Mikrokontroler ESP32 merupakan platform komputasi *edge* berbiaya rendah dengan arsitektur *dual-core* yang didukung oleh sistem operasi *real-time* (FreeRTOS) untuk mengelola *multitasking* akuisisi data dan eksekusi kendali secara efisien [10]. Konsep modularitas pada *firmware* menerapkan pilar-pilar utama OOP—seperti abstraksi, enkapsulasi, dan polimorfisme—guna memisahkan logika program utama dari implementasi spesifik *driver* perangkat keras. Setiap komponen sensor dan aktuator diisolasi ke dalam antarmuka terabstraksi (*abstract interface*) yang bertindak sebagai kontrak dasar bagi seluruh *handler* protokol komunikasi [3].

Untuk mendukung fleksibilitas konfigurasi tanpa perlu melakukan kompilasi ulang kode (*recompilation*), arsitektur *firmware* memanfaatkan dua pola desain kreasional dan struktural utama:

- *Factory Pattern*: Merupakan pola desain kreasional (*creational pattern*) berbasis OOP yang mengabstraksi proses instansiasi objek. *Factory* bertanggung jawab membuat instansi objek *handler* secara dinamis berdasarkan parameter masukan saat *runtime*, sehingga *main loop* program tidak perlu mengetahui tipe objek konkret yang sedang dieksekusi [3].
- *Protocol Registry*: Merupakan pola struktural yang berfungsi sebagai struktur data terpusat (*registry*) untuk memetakan nama atau identifikasi protokol (seperti GPIO, Modbus RS485, atau I2C) ke *factory handler* yang sesuai. Pemetaan ini memungkinkan sistem mencocokkan konfigurasi berbasis teks (seperti berkas JSON atau parameter *Captive Web Portal*) dengan kelas *driver* yang tepat [3].

Kombinasi prinsip OOP, *Factory Pattern*, dan *Protocol Registry* menghasilkan arsitektur *firmware* berbasis konfigurasi (*configuration-driven firmware*).

2.5 Intelegensi Buatan untuk Kontrol Adaptif

Penerapan Inteligensi Buatan (*Artificial Intelligence*) pada sistem terdistribusi bertindak sebagai mekanisme pengambilan keputusan otonom (*autonomous decision-making*) yang menggantikan logika kontrol konvensional berbasis aturan kaku (*rule-based*). Domain kendali fisik bertumpu pada dua cabang utama:

- **Reinforcement Learning (TD3):** *Twin Delayed Deep Deterministic Policy Gradient* (TD3) merupakan algoritma *Deep Reinforcement Learning* (DRL) berbasis *actor-critic* yang beroperasi pada ruang keadaan (*state*) dan aksi (*action*) kontinu. TD3 dirancang untuk mengatasi masalah overestimasi nilai *Q-value* pada algoritma deterministik konvensional melalui tiga prinsip utama: pengoperasian *twin critics* (mengambil nilai minimum dari dua jaringan estimasi), pembaruan kebijakan yang tertunda (*delayed policy update*), dan *target policy smoothing*. Mekanisme ini memungkinkan pemodelan kontrol adaptif yang stabil terhadap dinamika variabel lingkungan fisik [5].
- **Visi Komputer (YOLOv8):** *You Only Look Once* versi 8 (YOLOv8) adalah arsitektur *deep learning* untuk deteksi objek dan segmentasi citra secara *real-time*. Berbeda dengan metode *two-stage detector*, YOLOv8 memprediksi lokasi *bounding box* dan probabilitas kelas dalam satu tingkatan pemrosesan jaringan saraf konvolusional (*convolutional neural network*). Fitur ini secara teoritis digunakan untuk ekstraksi fitur spasial/morfologi objek dari data spasial/citra secara efisien [11].

Dalam arsitektur terdistribusi, model inferensi AI/ML diisolasi ke dalam *service* independen yang terhubung via antarmuka komunikasi *asinkron* atau REST API. Landasan ini menerapkan prinsip *loose coupling*, di mana komputasi intensif model AI tidak memblokir (*non-blocking*) pipa pemrosesan data maupun alur eksekusi layanan inti sistem.

2.6 Pengujian Kinerja dan Keandalan Sistem Terdistribusi

Evaluasi sistem terdistribusi bertumpu pada kriteria keberhasilan yang mengukur dua aspek kritis: efisiensi pemrosesan data (*kinerja*) dan kemampuan mempertahankan operasional saat terjadi gangguan (*keandalan/resilience*) [4].

- **Evaluasi Kinerja Kuantitatif (*Performance Metrics*):** Landasan pengujian kinerja terdistribusi berfokus pada analisis beban (*stress testing*) untuk mengukur kemampuan tumpukan sistem dalam mengelola beban kerja puncak (*peak load*). Metrik dasar yang digunakan meliputi:
 - *Throughput*: Volume transaksi, permintaan per detik (*Requests Per Second / RPS*), atau pesan yang berhasil diproses oleh *event broker* dan *microservice* dalam satu satuan waktu [4].
 - *Latensi End-to-End*: Waktu tempuh total yang dibutuhkan data dari proses akuisisi di tingkat *edge*, transmisi jaringan, pemrosesan *microservice*, hingga penyimpanan basis data (diukur pada persentil ke-95 / P95 untuk merepresentasikan responsibilitas kondisi terburuk) [4].
 - *Utilisasi Sumber Daya*: Persentase konsumsi *Central Processing Unit* (CPU) dan *Random Access Memory* (RAM) pada tingkat kontainer guna mengidentifikasi titik kemacetan pemrosesan (*bottleneck*) [4].
- **Evaluasi Keandalan Arsitektur (*Chaos Engineering*):** *Chaos Engineering* merupakan metodologi empiris untuk menguji ketahanan sistem terdistribusi melalui penyuntikan gangguan secara sengaja (*fault injection*) pada lingkungan produksi atau beban penuh. Evaluasi ini bertujuan untuk memverifikasi batas isolasi kegagalan (*failure domain isolation*), memastikan bahwa kegagalan pada satu basis data atau *microservice* tidak memicu efek domino (*zero cascade failure*), serta menguji mekanisme pemulihan mandiri (*self-healing/auto-recovery*) antar-layanan terisolasi [6].

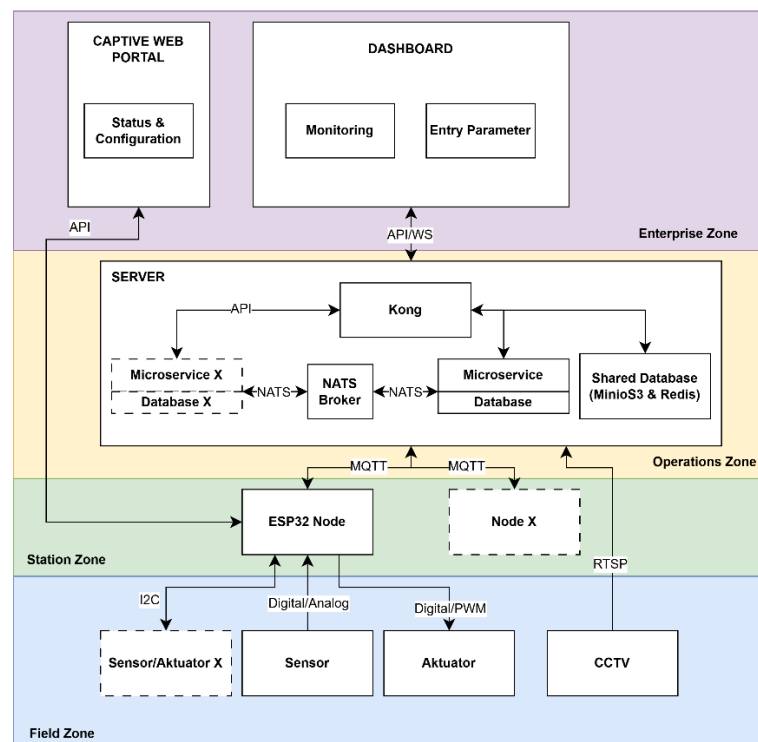
BAB III

PERANCANGAN SISTEM DAN METODOLOGI

Bab ini menguraikan metodologi dan tahapan perancangan sistem dengan menerapkan paradigma modularitas ganda pada lapisan *edge* dan *server*. Pendekatan ini dirancang untuk memberikan keberlanjutan dari dua perspektif utama: kemudahan bagi **pengembang** dalam memperbarui, mengonfigurasi, dan mengembangkan fitur sistem secara fleksibel (*scalability & maintainability*), serta kemudahan bagi **pengguna umum** (seperti mahasiswa dan petani) dalam mengoperasikan, memantau, dan mengelola sistem akuisisi data dan kontrol aktuator secara intuitif tanpa kendala teknis yang rumit (*usability*). Sebagai penutup, bab ini menyajikan skenario pengujian komprehensif untuk memvalidasi performa komputasi, keandalan sistem, dan responsivitas operasional di lapangan.

3.1 Arsitektur Sistem

Sistem ini dirancang dengan arsitektur terdistribusi dengan pembagian 4 batas zona fungsional yaitu *Field Zone*, *Station Zone*, *Operations Zone*, dan *Enterprise Zone*.



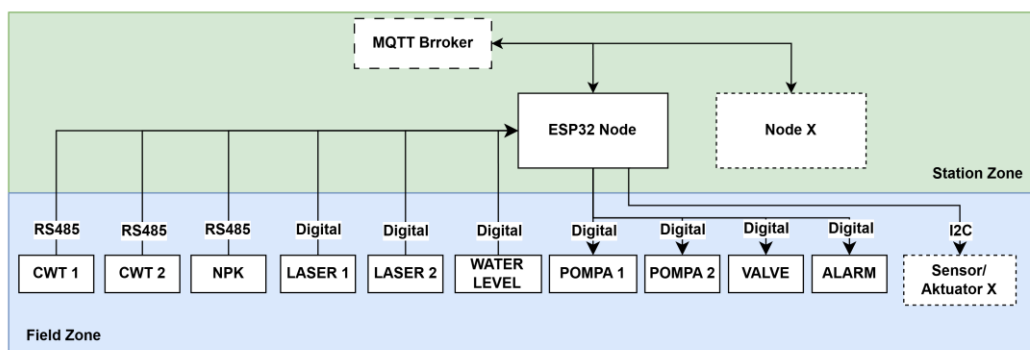
Gambar 3.1 General System Architecture Diagram

Gambar 3.1 merupakan ilustrasi arsitektur sistem secara umum. Untuk pembagian fungsi dari keempat zona fungsional tersebut adalah sebagai berikut:

1. **Field Zone:** Bertujuan untuk memantau dan memanipulasi kondisi lingkungan fisik, yaitu *module* aeroponik, dengan memanfaatkan berbagai sensor pengukur dan aktuator.
2. **Station Zone:** Berfungsi sebagai perantara komputasi tepi (*edge*) untuk akuisisi data dari sensor di lapangan dan mendistribusikan perintah aktuasi ke perangkat aktuator.
3. **Operations Zone:** Bertugas sebagai pusat komputasi yang menjalankan seluruh logika operasi sistem. Zona ini dibangun di atas arsitektur *microservice*, di mana setiap layanan memiliki tugas yang spesifik.
4. **Enterprise Zone:** Berperan sebagai antarmuka (*interface*) utama bagi pengguna akhir untuk berinteraksi, mengonfigurasi, dan memantau sistem.

3.2 Arsitektur Field Zone dan Station Zone

Arsitektur Field Zone berfokus kepada perangkat fisik yang terhubung dengan node ESP32 seperti sensor dan aktuator.



Gambar 3.2 Sensor and Actuator Configuration

Gambar 3.2 merupakan ilustrasi arsitektur sistem pada Field Zone. Konfigurasi perangkat keras pada sistem aeroponik ini mengintegrasikan 6 unit sensor mencakup sensor CWT, NPK, laser, dan *water level* serta 4 unit aktuator yang terdiri atas dua pompa, satu katup (*valve*), dan satu alarm. Sensor dan aktuator

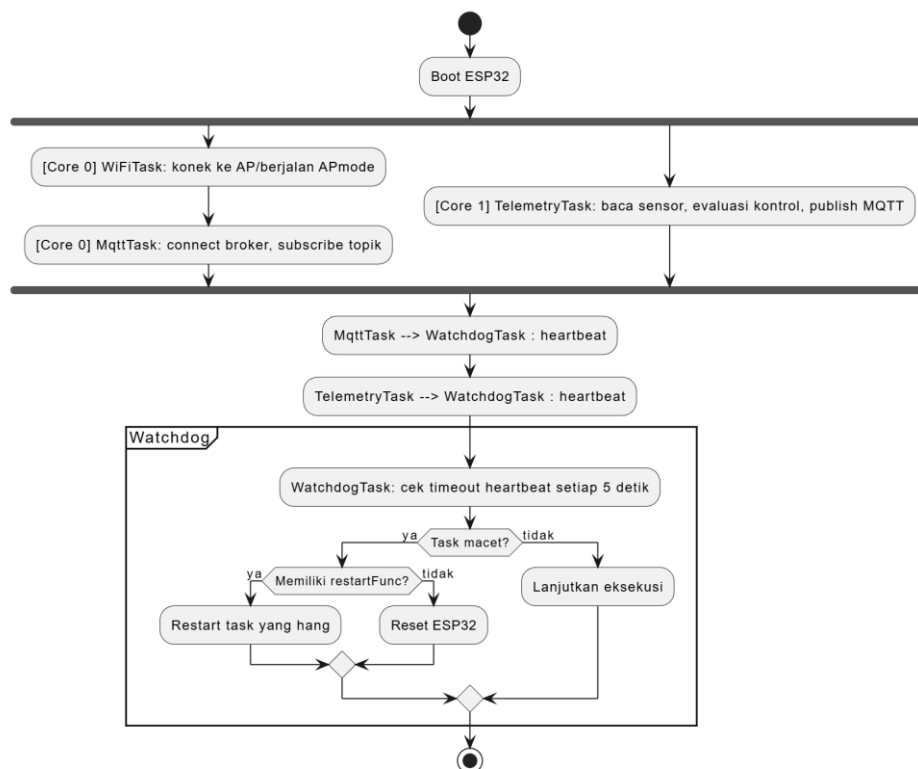
terhubung dengan mikrokontroler ESP32 yang sebagai *station* untuk mengelola pembacaan sensor dan perintah aktuator. Fokus utama pengembangan terletak pada aspek modularitas perangkat keras yang mempermudah penambahan sensor maupun aktuator baru secara dinamis tanpa membuat program dari awal.

3.3 Perancangan Firmware Modular ESP32

Firmware ESP32 dirancang diatas FreeRTOS dengan pendekatan *Protocol Registry* dan *Factory Pattern*. Dimana konfigurasi ini memungkinkan sensor atau aktuator baru cukup didaftarkan via file konfigurasi, tanpa mengubah inti program.

3.3.1 FreeRTOS

Desain ini memanfaatkan kinerja *dual-core* ESP32 yang bertujuan untuk membagi beban komputasi secara efisien dan memastikan keandalan eksekusi tugas fisik maupun komunikasi tanpa adanya pemblokiran (*non-blocking*).



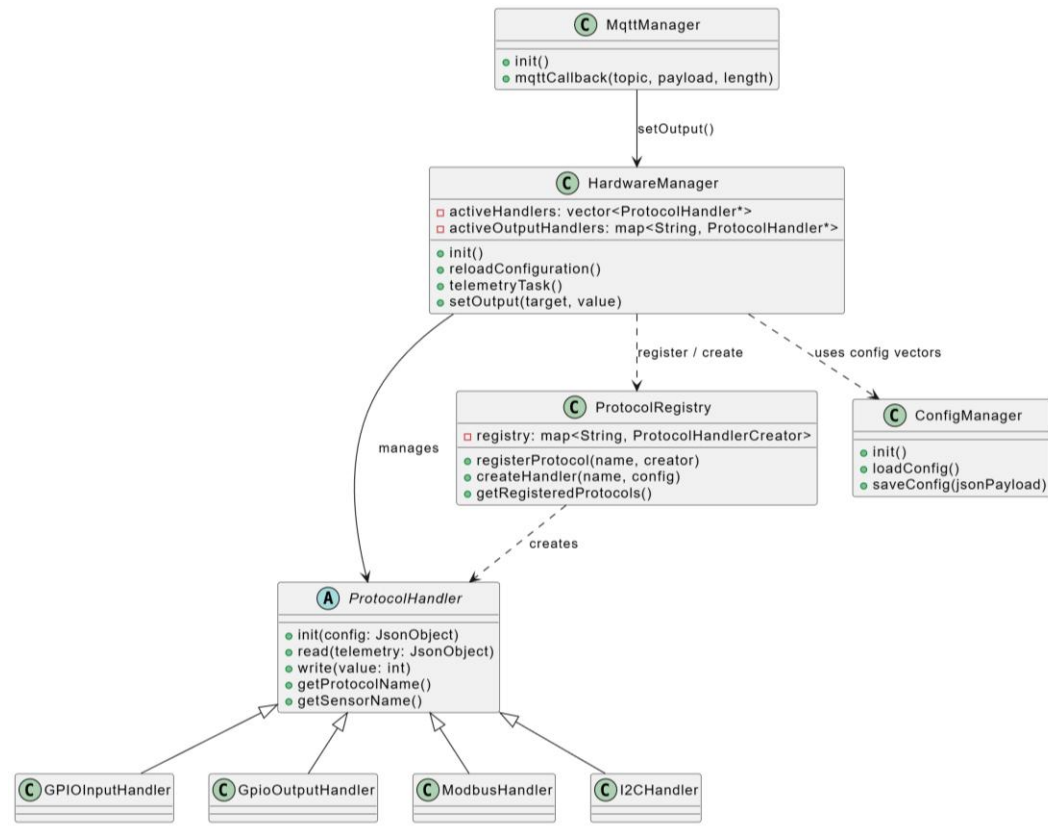
Gambar 3.3 Activity Diagram - Core Runtime ESP32 FreeRTOS

Gambar 3.3 merupakan diagram yang akan menjelaskan bagaimana aktifitas FreeRTOS di dalam perangkat. Core 0 menangani semua komponen jaringan dan pengawasan sistem, yaitu WiFiTask untuk menjaga koneksi internet dan melayani *Captive Web Portal*, MqttTask untuk mengelola komunikasi dengan broker MQTT dan menerima perintah aktuator, serta WatchdogTask yang berfungsi sebagai pengawas kesehatan seluruh task. SysMonitorTask untuk memantau penggunaan memori dan DiscoveryPeriodic yang secara otomatis dibuat oleh MqttTask untuk mengirimkan pesan discovery setiap 60 detik guna memastikan Module Service selalu mengetahui node yang aktif.

Core 1 dialokasikan untuk TelemetryTask yang bertugas membaca sensor secara periodik setiap 5 detik (*default*), memproses data GPIO, Modbus RS485, dan I2C, lalu memformatnya menjadi JSON sebelum diteruskan ke MqttTask untuk dipublikasikan. Hanya MqttTask dan TelemetryTask yang mengirim *heartbeat* ke WatchdogTask. Jika salah satu task berhenti mengirim *heartbeat*, watchdog akan *me-restart task* tersebut atau memicu reset sistem, sehingga kegagalan satu komponen tidak memblokir operasi lain.

3.3.2 Modular I/O : Factory Pattern & Registry

Untuk mendukung modularitas sistem I/O, digunakan kombinasi *Factory Pattern* dan *Registry* yang memisahkan logika manajemen perangkat keras dari protokol komunikasi. Rancangan struktur kelas dan alur pendaftaran antarmuka handler tersebut digambarkan pada Gambar 3.4.



Gambar 3.4 Class Diagram - Factory Pattern & Registry ProtocolHandler

ConfigManager memuat config.json dari LittleFS saat boot dan memetakan elemen JSON ke dalam std::vector in-memory di namespace Config: HardwareInputs, HardwareOutputs, HardwareModbus, HardwareSensors, dan LocalControlRules. Di config.json, protokol didefinisikan hanya sebagai string seperti "GPIO", "MODBUS", "I2C", atau "GPIO_OUT". Untuk kode program config.json dan ConfigManager terdapat pada Lampiran B.1 dan Lampiran B.2.

Saat HardwareManager::init() dijalankan, semua handler didaftarkan ke ProtocolRegistry sebagai peta String → lambda creator. Setiap kali konfigurasi diubah melalui *Captive Web Portal*, reloadConfiguration() memanggil ProtocolRegistry::createHandler() untuk membuat instance handler sesuai nama protokol di JSON, lalu menyimpannya ke activeHandlers (sensor) atau activeOutputHandlers (aktuator). Kode program reloadConfiguration() dapat dilihat pada Lampiran B.7.

TelemetryTask mengiterasi activeHandlers dan memanggil read() untuk setiap sensor, sedangkan MqttCallback menerima perintah aktuator dan memanggil HardwareManager::setOutput() yang pada akhirnya memanggil write() pada handler aktuator yang sesuai.

3.3.3 Vector Registry

Vector Registry merupakan mekanisme penyimpanan konfigurasi sensor ke dalam sebuah *list* (vektor) yang tersusun berurutan. Pendekatan ini dipilih karena TelemetryTask membaca seluruh sensor secara berkala dalam satu siklus. Dengan vektor, proses pembacaan cukup dilakukan dengan iterasi sekuensial firmware berjalan dari elemen sensor pertama hingga terakhir. Dengan menggunakan metode ini pembacaan sensor dapat berjalan lebih efisien dan memiliki urutan telemetry yang konsisten.

Kode pada Lampiran B.8 adalah implementasi dari vector registry untuk activeHandlers yang merupakan array dinamis untuk menampung pointer ke objek ProtokolHandler. Karena tipenya ProtokolHandler* (pointer ke *base class*), vektor ini dapat menyimpan handler dari berbagai jenis sekaligus GPIOInputHandler, ModbusHandler, I2CHandler berkat sifat *polymorphism*. Jadi nantinya activeHandlers akan berisi semua sensor yang aktif, apapun protokol komunikasinya.

Selanjutnya adalah membangun handler yang akan menangani protokol komunikasi langsung ke sensor. Seluruh kode program handler dapat dilihat pada Lampiran B.10 – Lampiran B.13.

Sebagai contoh fungsi ModbusHandler::read yang dapat dilihat pada Lampiran B.11 menangani akuisisi data sensor Modbus RTU melalui antarmuka serial RS485. Alur kerja dimulai dengan mengamankan bus serial menggunakan mekanisme saling-pengecualian (*mutex*) modbusMutex demi mencegah tabrakan data (*collision*), dilanjutkan dengan penyesuaian kecepatan transmisi (*baud rate*) secara dinamis. Melalui iterasi daftar konfigurasi, fungsi ini membaca *Input Register* atau *Holding Register* sesuai parameter target, lalu mengalikan nilai mentah hasil pembacaan dengan faktor pengali (*multiplier*). Hasil akhirnya

kemudian dipetakan secara terstruktur ke dalam sub-objek "modbus" pada *payload* JSON telemetri sekaligus disimpan pada *local cache* (latestSensorValues) untuk kebutuhan kendali lokal secara waktu nyata (*real-time*).

Sebagai ilustrasi, apabila sensor dengan parameter nama perangkat "soil_sensor" mengidentifikasi register kelembapan "humidity" dengan nilai terkalibrasi 45.5, maka struktur luaran JSON akan seperti pada Lampiran B.12

3.3.4 Penambahan Protokol Baru I2C

Ketika ingin menambahkan protokol baru seperti I2C maka dibuat fungsi I2CHandler::read pada sensor bertipe INA219 menangani akuisisi parameter kelistrikan sistem melalui bus komunikasi I2C. Alur kerja dimulai dengan menginisialisasi sub-objek nama perangkat (seperti "power_monitor") di dalam objek "i2c" pada JSON telemetri. Program kemudian mengukur nilai tegangan bus (*bus voltage*), tegangan *shunt*, arus beban (*current*), dan konsumsi daya (*power*) secara langsung dari modul sensor. Parameter tersebut dipetakan ke dalam *payload* JSON telemetri sekaligus disimpan pada *local cache* (latestSensorValues) guna mendukung kebutuhan pemantauan dan manajemen daya secara waktu nyata (*real-time*). Untuk panduan konfigurasi penambahan protokol baru dapat dilihat pada Lampiran B.20.

3.3.5 Map Registry

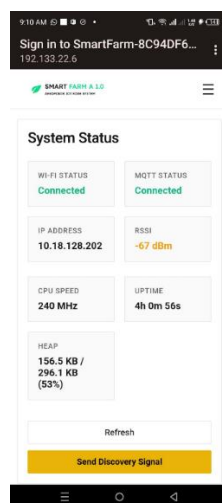
Berbeda dengan komponen sensor yang disimpan menggunakan struktur std::vector untuk kebutuhan iterasi secara massal (batch), modul aktuator menerapkan struktur peta pencarian (map) activeOutputHandlers, inisiasi *Map Registry* dapat dilihat pada Lampiran B.14. Pemilihan struktur map ini didasarkan pada kebutuhan pencarian leksikal cepat dengan kompleksitas algoritma $O(\log n)$ tanpa melalui proses perulangan (looping) saat fungsi setOutput() menerima parameter identitas aktuator. Sama dengan sensor, aktuator mengimplementasikan abstraksi dari kelas induk ProtocolHandler dan mendaftarkan kelas konkretnya pada ProtocolRegistry, namun dengan penambahan kontrak metode virtual murni write(int value) untuk menangani instruksi penulisan status fisik perangkat secara terisolasi.

Ketika MqttManager menerima pesan instruksi dari server melalui topik smartfarm/actuator/<node_id> dengan payload JSON seperti pada **Lampiran B.16**. Fungsi pendengar mqttCallback segera merespons dengan mengurai (parsing) muatan JSON tersebut menjadi tiga parameter operasional utama, yaitu tindakan (action = "set_output"), target aktuator (target = "load1"), dan nilai keluaran (value = 0).

Ketiga parameter ini kemudian diteruskan ke fungsi setOutput(targetName, value) kode dapat dilihat pada Lampiran B.16. Jika target "load1" ditemukan di dalam peta registrasi activeOutputHandlers, fungsi akan mengeksekusi metode write(0), memperbarui status operasional terakhir pada registri outputStates, serta mengirimkan sinyal interupsi RTOS melalui instruksi xTaskNotifyGive(telemetryTaskHandle) guna memicu tugas TelemetryTask agar segera mengirimkan laporan pembaruan status aktuator ke server secara instan.

Kelas GpioOutputHandler::write kode program dapat dilihat pada Lampiran B.17 mengeksekusi perintah berdasarkan jenis konfigurasi pin aktuator. Apabila dikonfigurasi sebagai PWM (Pulse Width Modulation), sistem mengeksekusi perintah analogWrite dengan rentang value 0 sampai 255. Sebaliknya, jika dikonfigurasi sebagai DIGITAL, sistem mengeksekusi perintah digitalWrite (HIGH/LOW) untuk mengubah status kelistrikan pin GPIO secara langsung.

3.4 Antarmuka Firmware: Captive Web Portal



Gambar 3.5 Captive Web Portal Interface

Untuk mempermudah pengelolaan konfigurasi berkas `config.json`, sistem menyediakan antarmuka yang dapat diakses melalui mekanisme *Captive Web Portal* seperti pada Gambar 3.5. Akses antarmuka dilakukan dengan menghubungkan perangkat pengguna ke titik akses nirkabel (*wireless access point*) yang dipancarkan oleh ESP32 dengan SSID SmartFarm-<node_id>.

3.5 Standarisasi Komunikasi MQTT

MQTT beroperasi sebagai satu-satunya jalur komunikasi yang menjembatani *Station Zone (edge)* dan *Operations Zone (server)*. Oleh karena itu standarisasi komunikasi MQTT diperlukan untuk efisiensi transfer data. Selain itu, standarisasi ini juga memastikan kemudahan integrasi dan pengembangan sistem di masa depan, sehingga berbagai jenis *node/edge* yang berbeda dapat tetap terhubung ke server menggunakan standar protokol yang sama.

3.5.1 Parameter dan Kontrak Topik

Parameter koneksi MQTT (seperti broker host, port, dan topic prefix) dapat dikonfigurasi via *Captive Web Portal*. Struktur topik MQTT menerapkan pola hierarki:

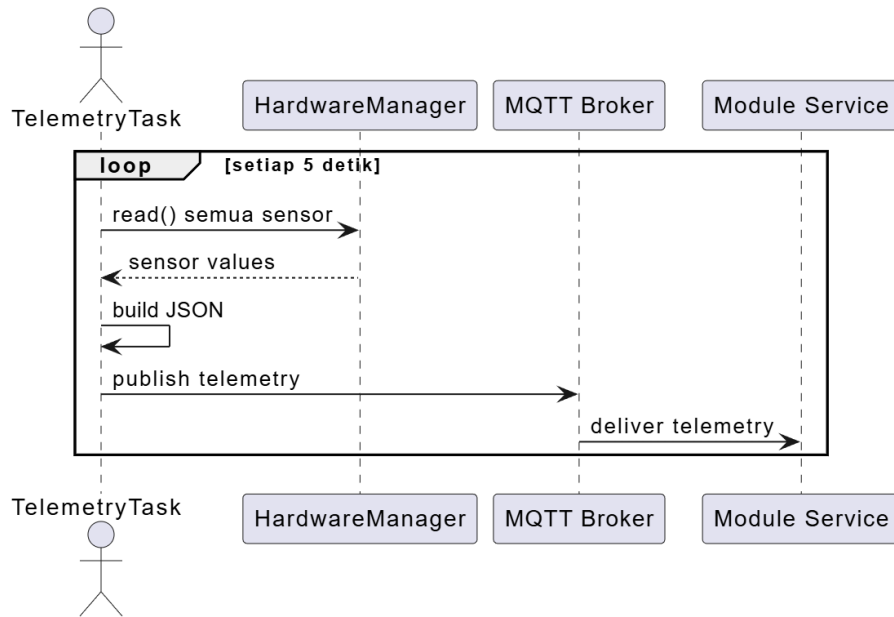
```
{prefix}/{domain}/{node_id}

#{prefix} = smartfarm (configurable via MQTT_TOPIC_PREFIX)
#{domain} = kategori pesan (telemetry, actuator, confirm, status, discovery, dll.)
#{node_id} = identitas unik perangkat (misalnya: node-01, node-02)
```

Pola ini memungkinkan penggunaan wildcard subscription di sisi server, seperti `smartfarm/+/telemetry` untuk memantau seluruh telemetry atau `smartfarm/actuator/+` untuk seluruh perintah kontrol. Detail lengkap mengenai kontrak topik terdapat pada **Lampiran C.1**.

3.5.2 Standar Output Telemetry dan Kendali Aktuator

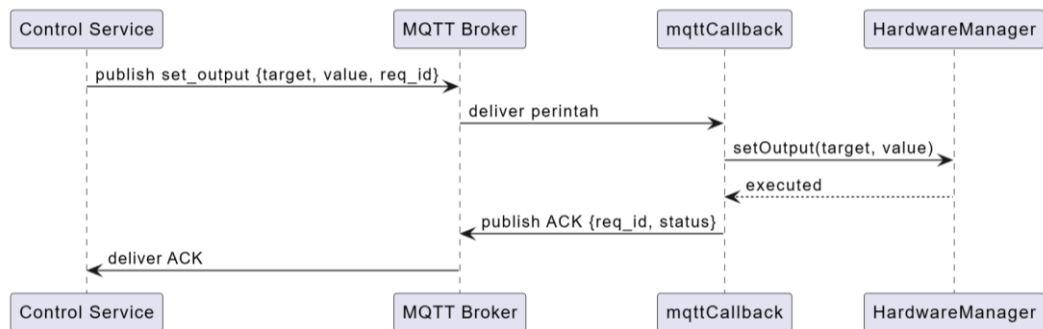
Gambar 3.6 merupakan gambaran bagaimana data telemetry dari sensor (*edge*) dikirimkan ke module service (*server*).



Gambar 3.6 Sequence Diagram — Telemetry Flow

Firmware mengirimkan data (smartfarm/{node_id}/telemetry) sensor fisik, metrik jaringan, dan status hardware ke Module Service (default 5 detik) dengan struktur JSON yang dapat dilihat pada **Lampiran C.2**.

Gambar 3.7 menjelaskan bagaimana Control Service mengirimkan perintah sampai ke aktuator.



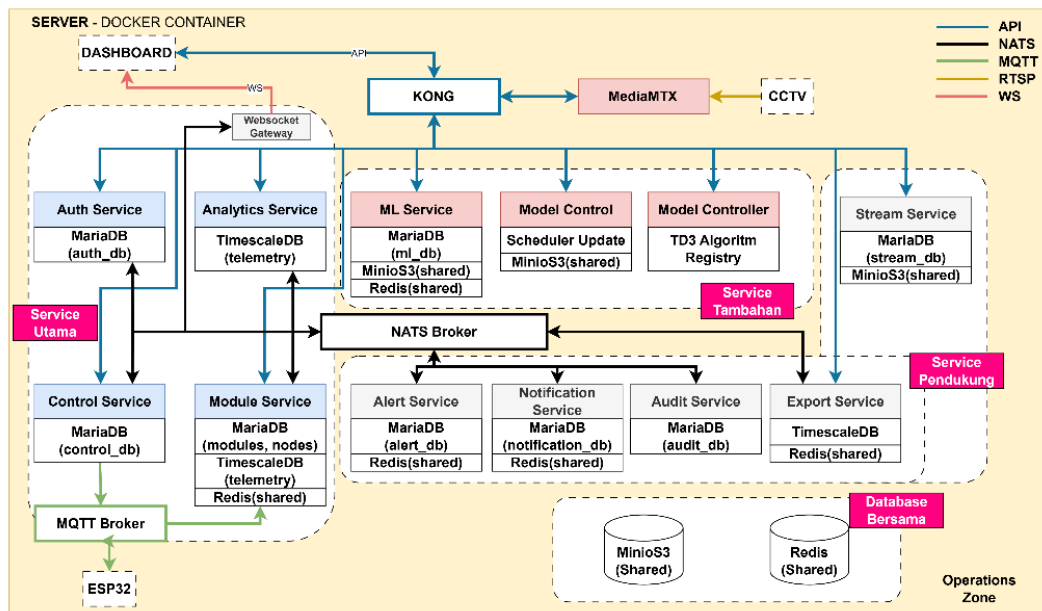
Gambar 3.7 Sequence Diagram — Actuator Command Flow

Control Service mengirim perintah ke topik smartfarm/actuator/{node_id} dengan struktur yang dapat dilihat pada **Lampiran C.3**. Kemudian perintah diterima oleh mqttCallback, diparse, kemudian diteruskan ke HardwareManager::setOutput() yang mencari handler aktuator di activeOutputHandlers berdasarkan nama target

dan memanggil `write(value)`. Setelah eksekusi berhasil, ESP32 mengirim ACK ke `smartfarm/{node_id}/confirm` dengan payload seperti pada Lampiran C.4.

3.6 Arsitektur Sistem Server

Data telemetri yang telah diakuisi oleh perangkat *edge* ditransmisikan menuju *server* menggunakan protokol komunikasi MQTT. Di sisi *server*, seluruh aliran data yang masuk dikelola secara terisolasi pada *Operation Zone* untuk memastikan proses ingesti, pemrosesan, dan penyimpanan data berjalan secara responsif. Arsitektur microservice yang diterapkan pada *Operation Zone* ditunjukkan pada Gambar 3.8 berikut.



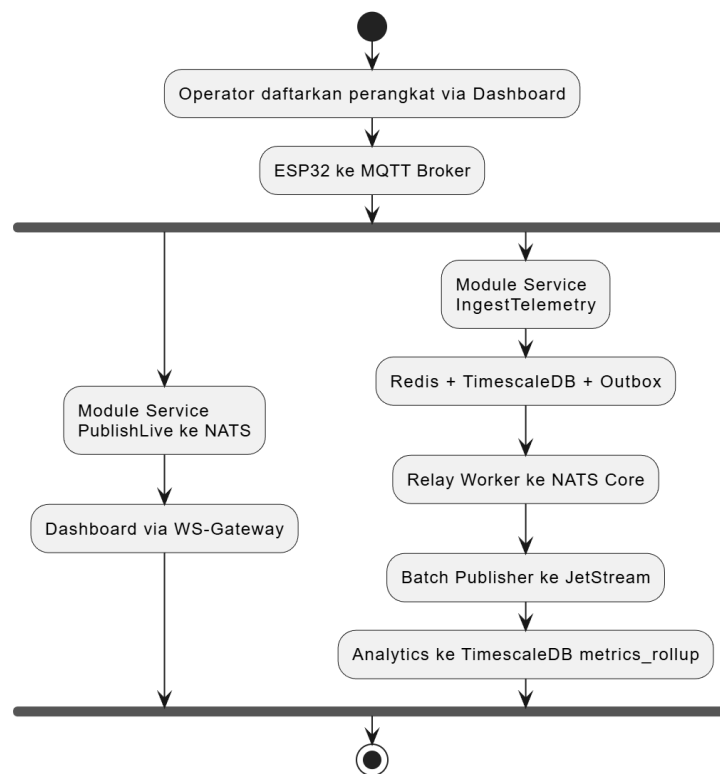
Gambar 3.8 Microservice Architecture Diagram

Terdapat 13 layanan microservice dengan 12 *service* yang menerapkan prinsip *database-per-service*, dengan total 12 *instance* database terpisah yang terdiri atas 8× MariaDB, 2× TimescaleDB, 1× Redis, dan 1× MinIO. Seluruh layanan terhubung melalui *event bus* NATS JetStream, REST API digunakan untuk komunikasi antar layanan eksternal, dan Websocket Gateway untuk komunikasi real-time, serta satu-satunya pintu masuk dari luar yaitu Kong Gateway. Rincian fungsi masing-masing layanan terdapat pada **Lampiran A.1**. Dari ke-13 layanan tersebut, ML Service, model-control, dan model-controller dikembangkan sebagai layanan eksternal yang berkomunikasi dengan layanan inti melalui komunikasi

eksternal API. Ketiga layanan ini bertujuan untuk mengendalikan aktuatur pompa *misting* dan *valve* secara adaptif.

3.7 Ingesti Data Telemetri Module Service

Module Service bertanggung jawab mengelola komunikasi MQTT langsung dengan ESP32 melalui akuisisi data telemetri dan status perangkat, melakukan manajemen modul serta registrasi node otomatis via sinyal *discovery*, menyimpan data *time-series* di database TimescaleDB, serta meneruskan seluruh *event* ke layanan *microservice* lain melalui NATS JetStream. Gambar 3.9 merupakan ilustrasi alur telemetri yang dikelola oleh *Module Service*.



Gambar 3.9 Data Flow Diagram — Telemetry Pipeline

Sebelum data telemetri dapat diolah oleh Module Service, perangkat perlu didaftarkan terlebih dahulu oleh operator melalui *dashboard* panduan konfigurasi dapat dilihat **Lampiran D.1**. Setelah ESP32 mempublikasikan data telemetri ke MQTT Broker, data tersebut diterima oleh fungsi `onMessage()` pada komponen `subscriber.go` dapat dilihat pada **Lampiran D.3**.

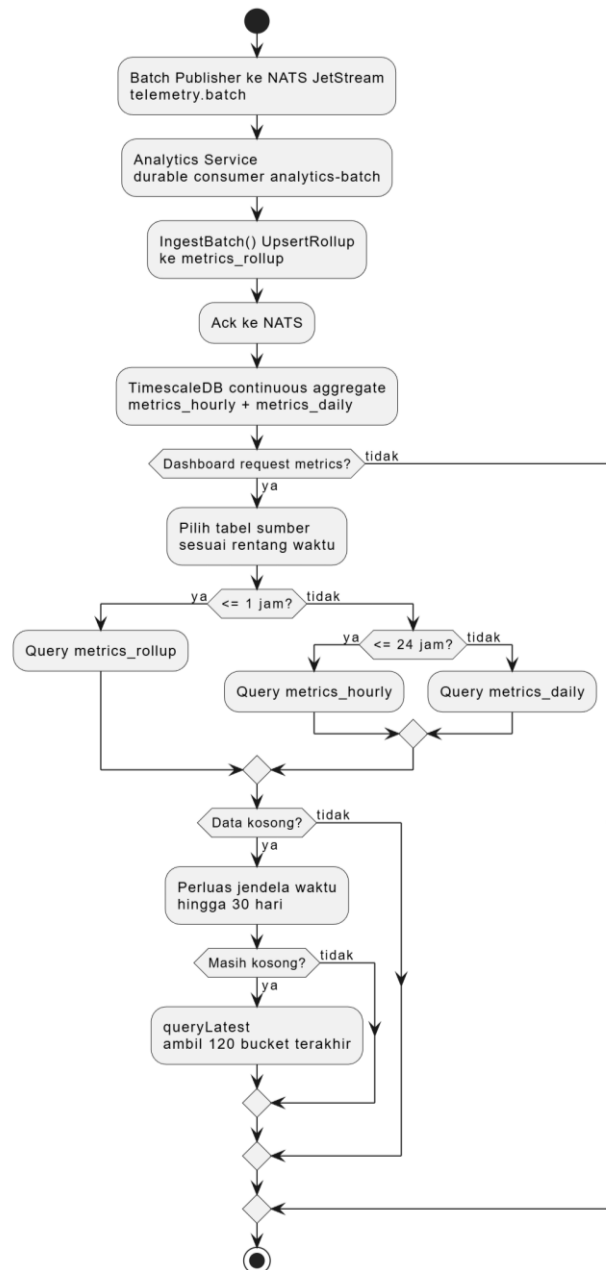
Fungsi `onMessage()` kemudian membagi alur pemrosesan data menjadi dua jalur:

1. Jalur Real-time (`PublishLive()`): Payload mentah dari ESP32 langsung diteruskan ke NATS (`mqtt.{node_id}`) untuk ditampilkan secara *real-time* pada *dashboard* pengguna melalui WebSocket-Gateway.
2. Jalur Ingest & Penyimpanan (`IngestTelemetry()`):
 - Data telemetry terlebih dahulu disimpan di Redis *cache* untuk pengaksesan cepat.
 - Selanjutnya, fungsi `IngestTelemetry()` pada **Lampiran D.4** melakukan ekstraksi nilai metrik dari payload JSON bersarang menggunakan aturan pemetaan (*tag mapping*) yang tersimpan pada *in-memory cache*. Pendekatan ini meniadakan *query* metadata ke MariaDB pada setiap pengiriman data sehingga proses akuisisi berjalan sangat cepat.
 - Nilai metrik numerik yang berhasil dipetakan kemudian ditulis langsung ke *hypertable* telemetry pada TimescaleDB sebagai data historis *time-series*.
 - Untuk menginformasikan kedatangan data ke layanan *backend* lainnya, sistem mengantrekan *event* dengan menuliskan catatan kejadian ke tabel *outbox* MariaDB dalam transaksi lokal.

Setelah proses ingest selesai, komponen **Relay Worker** mem-poll tabel *outbox* secara berkala untuk menerbitkan pesan ke NATS Core dengan *header* deduplikasi, menjamin prinsip *at-least-once delivery* meskipun *message broker* sempat terganggu. Di saat yang sama, **Batch Publisher** meng-agregasi telemetry per 1 menit ke NATS JetStream, yang selanjutnya dikonsumsi oleh **Analytics Service** untuk menyimpan hasil agregasi ke dalam tabel `metrics_rollup` di TimescaleDB.

3.8 Pemrosesan Data Analytics Service

Analytics Service merupakan layanan mikro yang bertugas mengolah dan mentransformasi data telemetry mentah dari *Module Service* menjadi informasi agregat statistik deret waktu (*time-series*) yang akan ditampilkan di *dashboard*. Gambar 3.10 dibawah ini merupakan gambaran alur kerja dari *Analytics Service*:



Gambar 3.10 Data Flow Diagram — Analytics Pipeline

Pesan telemetry `telemetry.batch` yang diterbitkan melalui NATS JetStream dikonsumsi oleh Analytics Service sebagai *durable consumer* `analytics-batch` menggunakan metode `DeliverAll()` dan `ManualAck`. Proses konsumsi ini berjalan secara independen dan asinkron, di mana setiap pesan diolah melalui fungsi `IngestBatch()` kode program dapat dilihat pada Lampiran E.3 dan di-*upsert* langsung ke tabel `metrics_rollup` pada database TimescaleDB (`timescaledb-analytics`) kode program dapat dilihat pada Lampiran E.4.

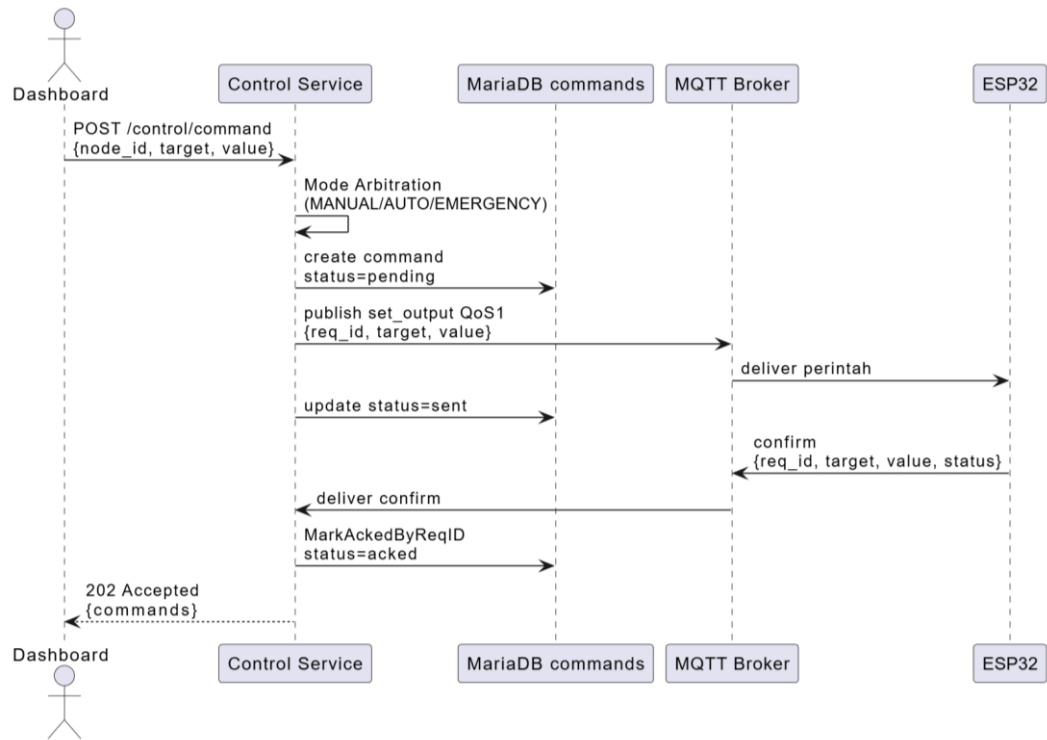
Data disimpan sementara oleh NATS dengan retensi 24 jam (`FileStorage`). Jika Analytics Service mengalami *down* atau *restart*, data yang belum terproses akan di-*replay* otomatis saat layanan kembali aktif. Sinyal pengakuan (*Ack*) hanya dikirim ke NATS setelah penyimpanan ke database berhasil. Jika terjadi kesalahan teknis saat menyimpan data, NATS akan mendistribusikan ulang (*redeliver*) pesan tersebut.

Untuk menghemat memori dan mempercepat waktu *query*, TimescaleDB secara otomatis mengagregasi data 1-menit dari `metrics_rollup` ke tabel agregasi bertingkat, yaitu `metrics_hourly` dan `metrics_daily`, melalui fitur *continuous aggregate*. Penggunaan tabel agregasi bertingkat ini bertujuan untuk memangkas ukuran *payload* dan mempercepat proses *rendering* grafik pada *dashboard* tanpa harus memproses ribuan titik data mentah.

Saat *dashboard* meminta data grafik (`GET /v1/analytics/metrics`), sistem secara otomatis memilih tabel sumber data berdasarkan rentang waktu yang diminta. Jika tidak ada data pada rentang waktu yang diminta, sistem memperluas jendela pencarian secara bertingkat hingga 30 hari. Apabila node sudah lama nonaktif dan data masih kosong, sistem menjalankan fungsi `queryLatest` untuk mengambil 120 *bucket* terakhir dari `metrics_rollup`. Mekanisme ini menjamin *dashboard* selalu menampilkan grafik atau titik data terakhir, bukannya grafik kosong.

3.9 Manajemen Kontrol Pada Control Service

Control Service bertindak sebagai *control plan* backend utama yang bertanggung jawab mengelola seluruh siklus hidup perintah aktuator. Gambar 3.11 merupakan gambaran alur kerja dari *Control Service*



Gambar 3.11 Sequence Diagram — Control Service Command

Setiap instruksi eksekusi kontrol bermuara pada fungsi `HandleManualCommand()` yang menjalankan komponen *Mode Arbitration* sebelum proses eksekusi seperti pada program Lampiran F.1. Arbitrase ini memvalidasi mode node saat ini via `GetNodeMode()` program terdapat pada Lampiran F.2 untuk mencegah *race condition* antara perintah manual dari dashboard, otomatisasi *scheduler*, dan kondisi darurat:

- **MANUAL:** Memprioritaskan intervensi pengguna. Perintah manual dapat dieksekusi, sementara *scheduler* internal secara otomatis menghentikan pengiriman instruksi karena fungsi `dispatch()` pada *scheduler* memvalidasi dan menolak eksekusi jika mode berada pada status MANUAL atau EMERGENCY Lampiran F.10.
- **AUTO:** Mode standar sistem terdapat pada Lampiran F.2 di mana *scheduler* memegang kendali penuh. Perintah manual dari *dashboard* akan ditolak dengan respons `ErrNodeAutoMode`, kecuali jika membawa parameter *bypass* khusus atau merupakan jenis perintah *emergency stop*.

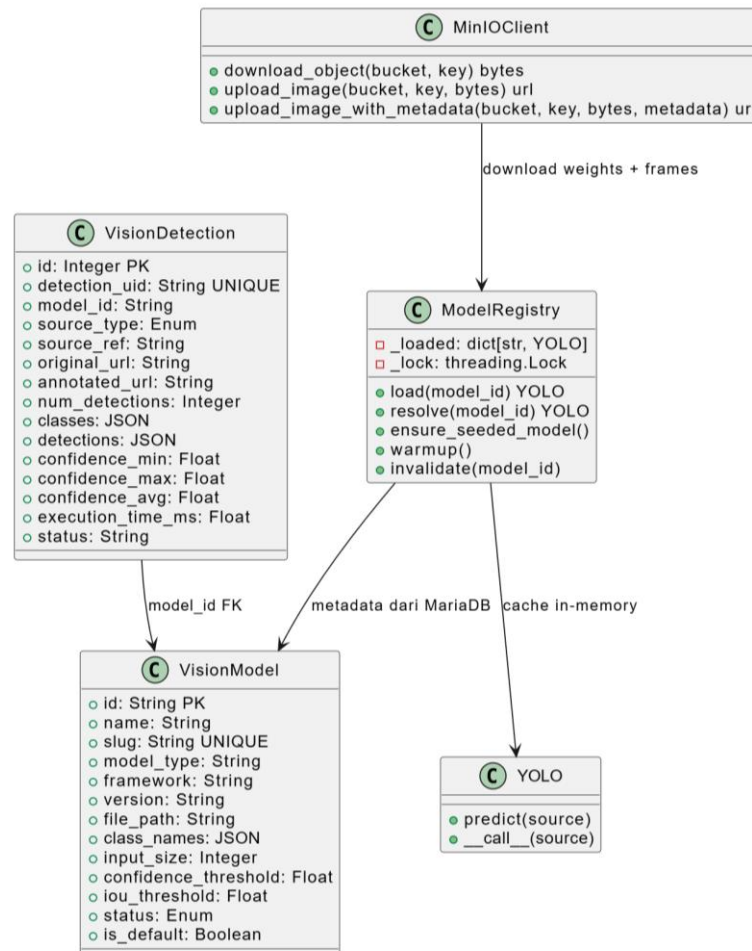
- **EMERGENCY:** Mengunci jalur eksekusi dan menolak seluruh instruksi kontrol umum (ErrNodeEmergency). Mode operasional sebelum darurat disimpan agar sistem dapat kembali ke kondisi normal secara aman.

Setelah lolos *arbitrase*, fungsi `dispatch()` pada *Control Service* memproses perintah ke ESP32 secara transaksional berbasis pesan `set_output` dengan jaminan Quality of Service (QoS) 1 via topik `smartfarm/actuator/{node_id}`. Seluruh alur ini terikat pada 5 tahapan status transaksional:

1. **pending:** Perintah dengan *UUID* `req_id` berhasil dibangkitkan dan dicatat ke basis data MariaDB melalui fungsi `CreateCommand()` seperti program pada Lampiran F.3.
2. **sent:** Fungsi `PublishSetOutput()` pada Lampiran F.4 mempublikasikan muatan JSON ke MQTT Broker. Jika sukses, status transaksi di MariaDB diperbarui menjadi `sent` via `UpdateCommandStatus()`.
3. **acked:** ESP32 membalas konfirmasi ke topik balasan. Komponen *Control Service* menangkap sinyal ini melalui fungsi `OnConfirm()`, lalu mengeksekusi `MarkAckedByReqID()` untuk mencocokkan `req_id` dan memperbarui status menjadi *acked*, kode program terdapat pada Lampiran F.5. Setelah itu, respons HTTP 202 Accepted dikembalikan ke *Dashboard*.
4. **timeout:** Jika dalam batas waktu tertentu konfirmasi tidak diterima, fungsi `TimeoutStale()` (*sweep worker*) secara otomatis menandai perintah sebagai timeout di basis data seperti pada program Lampiran A.6.
5. **failed:** Jika koneksi MQTT terputus (`!IsConnected()`) atau terjadi kesalahan saat mempublikasikan pesan, transaksi langsung dibatalkan dan statusnya diubah menjadi `failed`.

3.10 Penambahan Layanan Pengolahan Citra

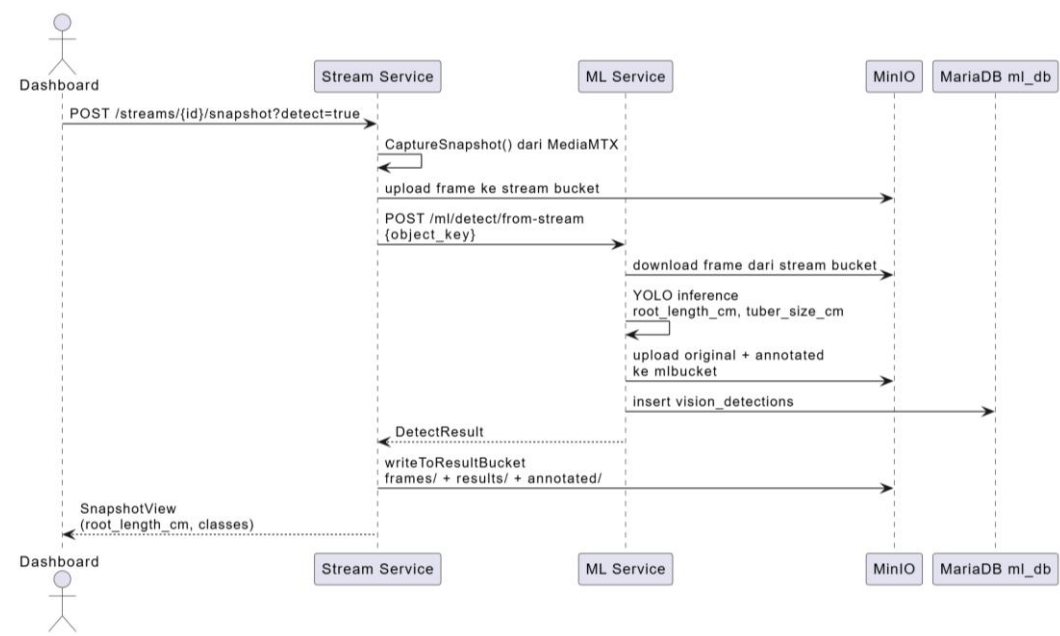
Untuk mekanisme deteksi akar dan umbi diperlukan model pengolahan citra. Model tersebut sudah tersedia dan berbasis YOLO (*You Only Look Once*) arsitektur *single-stage object detector* dari pustaka Ultralytics Model yang telah siap ini dikelola oleh ML Service, yang memiliki fungsi ganda sebagai penyimpan model dan penyedia API untuk mengeksekusi inferensi gambar. Disediakan juga *Environment Variable* seperti pada Lampiran G.1 untuk melakukan konfigurasi awal kalibrasi pixel/cm, bucket, dan konfigurasi lainnya sebelum layanan dijalankan. Struktur kelas dan relasi antarkomponen pada modul inferensi visi komputer (*ML Service*)—yang mengelola pengunduhan bobot model, pemetaan metadata, *caching in-memory*, hingga penyimpanan hasil deteksi ditunjukkan pada Gambar 3.12 berikut:



Gambar 3.12 Class Diagram — Model Registry

ModelRegistry mengelola siklus hidup model YOLOv8 dengan metadata yang tersimpan di MariaDB vision_models. Model di-load secara lazy dan di-cache dalam _loaded dict yang dilindungi threading.Lock. Saat model dipanggil, load() memeriksa cache; jika belum ada, bobot .pt diverifikasi di /app/models lalu diinisialisasi via YOLO(path). Pembaruan model menghapus cache lama (_loaded.pop(model_id)) agar model baru langsung aktif tanpa downtime. VisionDetection menyimpan hasil inferensi dengan URL gambar original dan annotated di MinIO, serta metadata deteksi seperti root_length_cm, tuber_size_cm, confidence, dan waktu eksekusi.

Mekanisme pemrosesan snapshot citra dan eksekusi inferensi deteksi objek—mulai dari pengambilan *frame* oleh *Stream Service*, pemrosesan model YOLO pada *ML Service*, hingga penyimpanan hasil pada MinIO dan MariaDB ditunjukkan pada Gambar 3.13 berikut:



Gambar 3.13 Sequence Diagram — AI Detect Inference Flow

AI Detect pada Live Dashboard memicu POST /streams/{id}/snapshot?detect=true. Stream Service menangkap frame dari MediaMTX, mengunggahnya ke MinIO bucket stream, lalu memanggil ML Service POST /ml/detect/from-stream dengan object_key. ML Service mengunduh frame, memuat model YOLOv8 via ModelRegistry (dengan cache in-memory), menjalankan inferensi untuk

mendeteksi akar dan umbi, mengukur panjang akar dalam cm, lalu menyimpan gambar original dan annotated ke MinIO mlbucket. Hasil deteksi juga disimpan ke MariaDB vision_detections. Stream Service kemudian menulis salinan frame, JSON hasil, dan gambar annotated ke mlbucket sebelum mengembalikan SnapshotView ke Dashboard.

3.11 Algoritma Misting Adaptif dan Layanan Kontrol Adaptif

Pengembangan layanan kontrol adaptif diawali dengan melatih model Reinforcement Learning kemudian pengembangan layanan kontrol adaptif.

3.11.1 Algoritma Misting Adaptif

Model berupa Reinforcement Learning berbasis algoritma TD3 (Twin Delayed Deep Deterministic Policy Gradient) disimulasikan dengan menggunakan pustaka Python Gymnasium. Lingkungan simulasi dirancang semirip mungkin dengan kondisi fisik aeroponik nyata, lengkap dengan pengkondisian cuaca ekstrem serta noise sensor untuk menguji ketahanan model. Pelatihan ini bertujuan agar sistem mampu mengoptimalkan dan memperbarui durasi penyemprotan nutrisi secara otomatis berdasarkan dinamika lingkungan.

Model TD3 bertindak sebagai pengambil keputusan yang membaca kondisi sistem dengan data 10 dimensi Ruang Keadaan (State Space) seperti pada Tabel 3.1.

Tabel 3.1 Ruang Keadaan (State Space) - 10D

Dimensi	Variabel	Sumber Data (Deployment)
1	L_root — panjang akar (cm)	MinIO metadata deteksi ML
2	U_status — skor kondisi tanaman [0,1]	MinIO metadata deteksi ML
3	T_in — suhu dalam ruang	Telemetry Module Service
4	H_in — kelembapan dalam ruang	Telemetry Module Service
5	T_out — suhu luar ruang	Telemetry Module Service
6	H_out — kelembapan luar ruang	Telemetry Module Service

7	EC — electrical conductivity	Telemetry Module Service
8	pH — keasaman larutan nutrisi	Telemetry Module Service
9	T_nut — suhu larutan nutrisi	Telemetry Module Service
10	I_day — indeks intensitas matahari	Kalkulasi berdasarkan jam lokal

Kemudian model secara dinamis menentukan 3 dimensi Ruang Aksi (Action Space), yaitu mengatur durasi pengkabutan (misting), menentukan interval jeda penyemprotan, serta mengendalikan status buka-tutup katup (valve) seperti pada Tabel 3.2.

Tabel 3.2 Ruang Aksi (Action Space) - 3D

Dimensi	Aksi Ternormalisasi [-1,1]	Rentang Fisik
a_mist	Durasi misting	D_mist dalam [120, 600] detik
a_interval	Jeda antar misting	interval_sec dalam [120, 600] detik
a_valve	Buka/tutup valve	A_valve: 0=OFF, 1=ON (threshold: nilai $\geq 0 \rightarrow$ ON)

Model TD3 dilatih menggunakan pendekatan **dense reward shaping** agar agent belajar mengambil keputusan kontrol yang optimal, seimbang, dan aman. Sinyal reward dirancang menjadi tiga kategori utama:

1. **Insentif Utama & Stabilitas Lingkungan:** Agent diberi reward berdasarkan pertumbuhan fisik tanaman yang dinormalisasi dengan kesesuaian lingkungan. Untuk mengatasi sparse reward, sinyal padat (*dense signal*) diberikan setiap langkah berbasis faktor pembatas lingkungan (pH, EC, kelembapan, suhu udara/akar, dan oksigen).
2. **Bonus Kelangsungan Hidup (Survival Bonus):** Agent mendapatkan bonus bertingkat seiring berjalannya waktu (+0,5/langkah hingga +200 pada milestone 18 jam). Jika episode mati prematur akibat kondisi kritis, agent dikenakan penalti keras.

3. **Efisiensi Sumber Daya & Keragaman Aksi:** Memberikan cost pada penggunaan valve serta insentif untuk durasi misting lebih singkat dan interval lebih panjang. Penalti diberikan jika aksi agent terlalu monoton (*mode collapse*) atau berisiko tinggi.

Fungsi reward total dirancang untuk memprioritaskan pertumbuhan tanaman, stabilitas lingkungan, efisiensi sumber daya, dan eksplorasi aksi yang beragam. Total reward dihitung sebagai:

$$\begin{aligned}
 R_{\{total\}} = & R_{\{growth\}} + R_{\{growth_proxy\}} + R_{\{state\}} + P_{\{diversity\}} + R_{\{efficiency\}} \\
 & - C_{\{resource\}} - P_{\{env\}} - P_{\{hypoxia\}} - P_{\{extreme\}} - P_{\{shrink\}} \\
 & - P_{\{death\}}
 \end{aligned}$$

Untuk detail lengkap penentuan reward setiap parameter terdapat pada Lampiran H.2.

Kemudian model dilatih sebanyak 2 juta *timesteps* menggunakan kapasitas *replay buffer* sebesar 2 juta data. Kompleksitas simulasi dinamika lingkungan aeroponik serta besarnya volume sampel pengalaman yang diproses dalam *buffer* membuat iterasi pelatihan ini memakan waktu lebih dari satu hari penuh. Yang menghasilkan dua artefak utama, yaitu bobot model TD3 yang telah teroptimasi (*aeroponic_td3.zip*) dan vektor normalisasi data (*vec_normalize*). Kemudian model tersebut dimuat oleh *model-controller* untuk mengeksekusi inferensi kontrol msiting secara presisi.

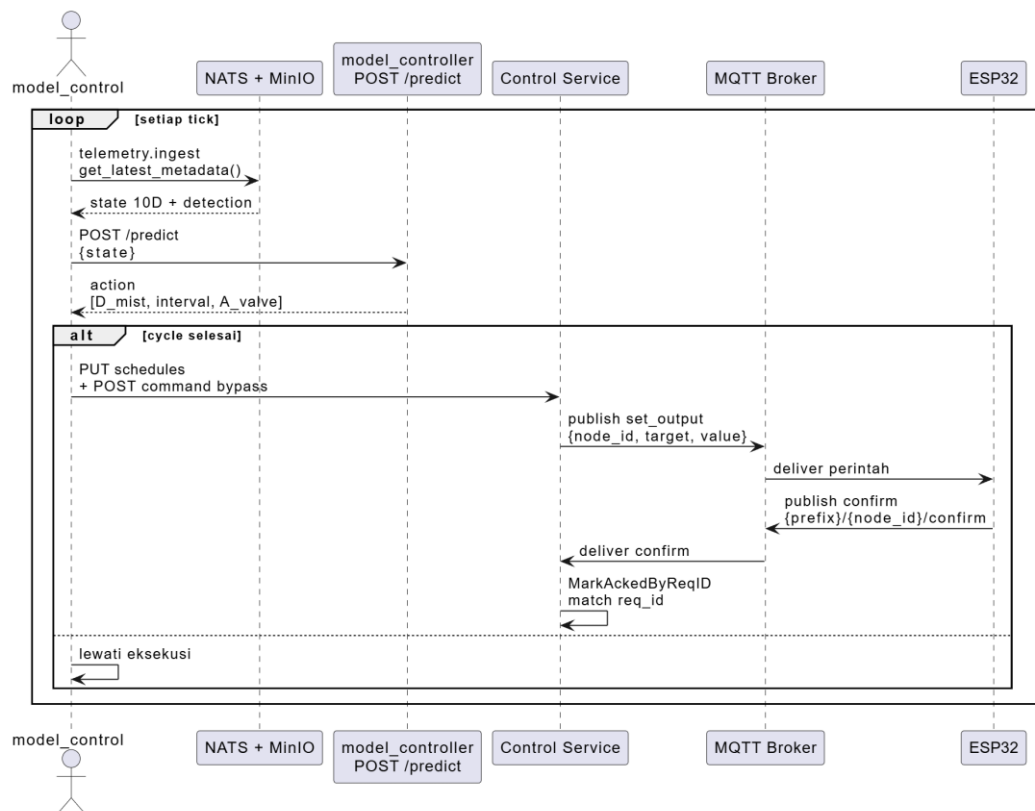
3.11.2 Layanan Kontrol Adaptif

Pada layanan kontrol adaptif mengintegrasikan dua layanan kustom baru untuk menguji algoritma kontrol alternatif: **model-control** dan **model-controller**. Kedua layanan ini bertindak sebagai entitas mandiri yang sepenuhnya terpisah dari sistem inti:

1. **model-control:** Layanan kustom berbasis Python yang mengelola pelatihan model pembelajaran penguatan (Reinforcement Learning) serta memproses keputusan logika kontrol berbasis AI secara otonom.

2. **model-controller**: Layanan perantara (*controller*) kustom yang bertindak sebagai jembatan untuk mengekspos API prediksi/kontrol model ke dunia luar dan berinteraksi dengan API Gateway Kong.

Alur interaksi inferensi *Reinforcement Learning* mulai dari ekstraksi state telemetry dan deteksi, prediksi aksi penyemprotan (POST /predict), hingga eksekusi perintah ke perangkat ESP32 via *Control Service* ditunjukkan pada Gambar 3.14 berikut:



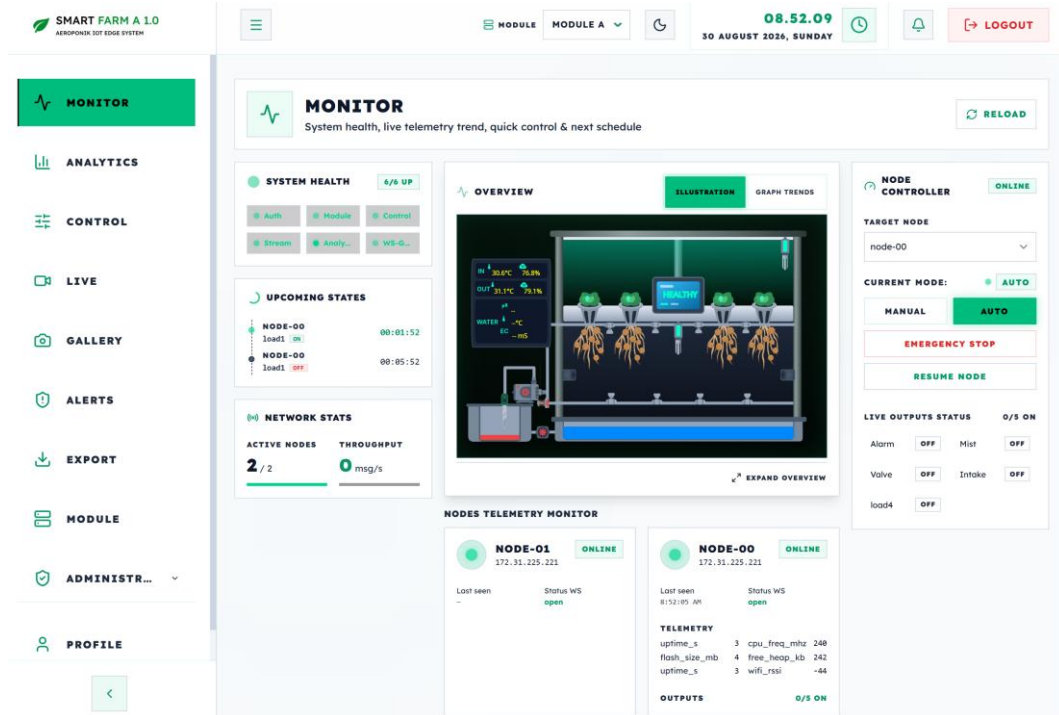
Gambar 3.14 Sequence Diagram — Adaptive Control Flow

Berikut alur singkat kontrol adaptif dari MinIO/telemetry hingga ESP32 dengan kode program yang dapat dilihat pada Lampiran I:

1. Data dikumpulkan baik telemetry ataupun metadata dari gambar model-control subscribe ke subject NATS telemetry.ingest untuk menerima data sensor real-time dari Module Service (suhu, kelembapan, EC, pH, dll.) model-control membaca metadata deteksi YOLOv8 (root_length_cm, condition) yang disimpan oleh ML Service di MinIO mlbucket/detected/.

2. Kemudian keseluruhan data disusun menjadi vektor state 10 dimensi:
[L_root, U_status, T_in, H_in, T_out, H_out, EC, pH, T_nut, I_day]
3. Selanjutnya model-control memanggil model-controller untuk Inferensi TD3 via POST /predict secara periodik (setiap 5 detik) dengan state 10D. model-controller mengembalikan aksi D_mist: durasi misting (120–600 detik), interval_sec: jeda antar misting (120–600 detik) dan A_valve: kontrol valve ON/OFF
4. Aksi baru disimpan sebagai pending_action. model-control mengecek apakah siklus pompa aktif saat ini sudah selesai. Jika ya, aksi diterapkan; jika tidak, siklus berjalan sampai selesai untuk mencegah mid-cycle reset.
5. Ketika pending sudah selesai model-control memperbarui jadwal via Control Service API (POST /v1/control/schedules). Juga model-control mengirim perintah langsung via Control Service API untuk kontrol valve.
6. Kemudian Control Service mempublikasikan perintah MQTT ke topik {prefix}/actuator/{node_id}.
7. Terakhir perintah diterima dan dieksekusi oleh ESP32 kemudian mengirim feedback konfirmasi ke {prefix}/{node_id}/confirm.

3.12 Dashboard



Gambar 3.15 Dashboard

Agar pengguna dapat berinteraksi dengan server dibuat antarmuka Dashboard seperti pada Gambar 3.15 agar mempermudah pengguna dalam monitoring maupun konfigurasi sistem. Semua data mengalir dari dashboard ke backend melalui Kong API dan WebSocket.

Berikut fitur yang terdapat pada Dashboard:

1. Real-time tanpa polling telemetry dan status di-push via WebSocket (WS-Gateway), sehingga grafik ikut bergerak saat data baru tiba.
2. Fitur utama monitoring (grafik telemetry hidup), analitik (riwayat & ekspor CSV), kontrol (perintah manual, jadwal otomatis, tombol emergency stop), live stream, galeri (snapshot/kamera), manajemen perangkat, dan manajemen profil
3. Peran (RBAC) admin/operator boleh mengontrol; viewer hanya boleh memantau analitik, galeri, dan profil (tidak bisa mengirim perintah).

Dengan pemisahan ini, nantinya developer dapat memperbaiki atau mengganti tampilan dashboard tanpa mempengaruhi sistem yang sudah berjalan di backend.

3.13 Skenario Pengujian

Pengujian performa sistem dilakukan secara bertahap dan progresif melalui skenario berikut:

1. Pengujian Modularitas Perangkat Tepi: Validasi asimilasi sensor/aktuator baru secara dinamis melalui *Captive Web Portal* tanpa proses *recompilation*.
2. Pengujian Integrasi *End-to-End*: Validasi kelancaran pipa data dari ESP32, Module Service, NATS JetStream, hingga visualisasi dasbor dan eksekusi kontrol.
3. Evaluasi Tahap 1 (Konfigurasi Minimal): Pengukuran performa dasar (baseline) jaringan dan komputasi tanpa beban algoritma kecerdasan buatan (AI).
4. Evaluasi Tahap 2 (Konfigurasi Penuh): Pengujian sistem di bawah beban komputasi AI penuh, mencakup inferensi YOLOv8 dan kontrol adaptif RL TD3 secara simultan.
5. Analisis Komparatif: Komparasi hasil kedua tahap pengujian untuk menganalisis dampak integrasi layanan tambahan terhadap *throughput*, latensi, dan stabilitas sistem.
6. Kesimpulan Awal: Sintesis hasil untuk memverifikasi efektivitas paradigma modularitas ganda terhadap kemudahan pengembangan dan keandalan otomatisasi sistem.

BAB IV

HASIL PENELITIAN

Bab ini menyajikan hasil implementasi, pengujian bertahap dan pembahasan komprehensif sistem otomasi aeroponik terdistribusi yang berfokus pada modularitas dan skalabilitas.

4.1 Pengujian Modularitas Sensor, Aktuator & Konfigurasi Dinamis via Antarmuka Web Captive Portal

Pertama perangkat ESP32 harus di-*flash* menggunakan versi *firmware* terbaru yang tersedia pada repositori GitHub untuk petunjuk *flashing* ada di Lampiran J.1. Proses pengunggahan ini dapat dilakukan menggunakan *Flash Tool* atau melalui fitur *Web Flash* pada [Espressif ESPTool Online](#).

4.1.1 Skenario Penambahan Sensor dan Aktuator

Pada tahap pengujian awal, *node* sensor ESP32 berhasil dikonfigurasi untuk mendukung berbagai jenis sensor, seperti digital, analog, Modbus, dan I2C. Selain itu, perangkat ini juga mampu mengendalikan aktuator menggunakan sinyal digital maupun PWM.

Pengujian dilakukan di dua lokasi berbeda dengan penyesuaian sebagai berikut:

- **Screenhouse:** Pengujian difokuskan pada lingkungan aeroponik menggunakan konfigurasi sensor dan aktuator yang relevan.
- **Lab PTIO ITB:** Pengujian dilakukan khusus untuk sensor I2C (INA219) guna mengukur parameter daya yang saat ini belum diterapkan di *screenhouse*.

Meski dilakukan di lokasi yang berbeda, kedua pengujian ini tetap konsisten karena menggunakan basis *firmware* yang sama. Perbedaan hanya terletak pada keberadaan modul sensor I2C.

Sebelum dilakukan pengujian perangkat harus dikonfigurasi terlebih dahulu agar terhubung ke broker untuk petunjuk terdapat pada Lampiran J.2. Selanjutnya pengguna dapat langsung melakukan konfigurasi sensor maupun aktuator.

Pada pengujian di lokasi pertama terdapat beberapa sensor digital dan modbus, untuk aktuator hanya digital berikut adalah hasil konfigurasi melalui antarmuka:

Input

input1
GPIO 32 | DIGITAL | Pull: NONE | IRQ: NONE | Debounce: 0ms [Edit] [Remove]

input2
GPIO 35 | DIGITAL | Pull: NONE | IRQ: NONE | Debounce: 0ms [Edit] [Remove]

input3
GPIO 34 | DIGITAL | Pull: NONE | IRQ: NONE | Debounce: 0ms [Edit] [Remove]

input4
GPIO 39 | DIGITAL | Pull: NONE | IRQ: NONE | Debounce: 0ms [Edit] [Remove]

GPIO Pin	Input Type	Pull Resistor	Interrupt Mode	Debounce (ms)
0	DIGITAL	NONE	NONE	0

Name Label:

Invert Logic: ☐ LOW = Active

[Done]

+ Add Input

Gambar 4.1 Konfigurasi Input GPIO

Output

load1
GPIO 25 | DIGITAL [Edit] [Remove]

load2
GPIO 26 | DIGITAL [Edit] [Remove]

load3
GPIO 27 | DIGITAL [Edit] [Remove]

load4
GPIO 14 | DIGITAL [Edit] [Remove]

buzzer
GPIO 33 | DIGITAL [Edit] [Remove]

GPIO Pin	Output Type	Name Label
0	DIGITAL	New Output

[Done]

+ Add Output

Gambar 4.2 Konfigurasi Output GPIO

Sensor Configurations

cwt1
ID: 1 | Baud: 9600 | Regs: 2 [Edit] [Remove]

cwt2
ID: 2 | Baud: 9600 | Regs: 2 [Edit] [Remove]

Sensor Name: New RS485 Sensor Slave ID: 1 Baudrate: 9600

Registers to Read

0	HOLDING	new_reg	1	[X]
---	---------	---------	---	-----

+ Reg

[Done]

+ Add Modbus Sensor

Save & Reboot

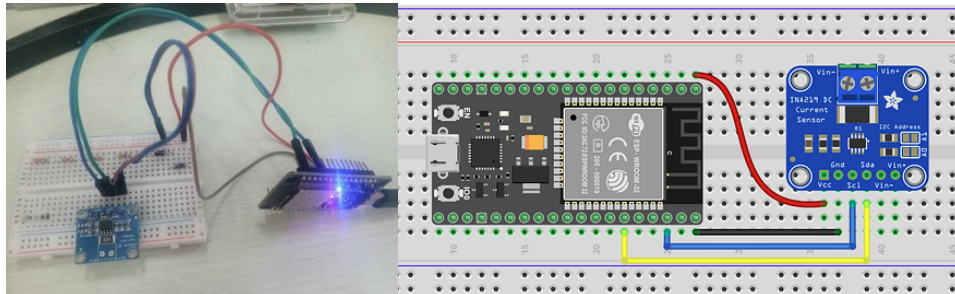
Gambar 4.3 Konfigurasi Sensor Modbus

Untuk pengujian pada sistem Aeroponik di Screenhouse dengan konfigurasi yang sama seperti pada Gambar 4.1 – 4.3 didapatkan data telemetri seperti berikut:

```
{
  "node_id": "ECE334219870",
  ...,
},
"telemetry": {
  "inputs": {
    "input1": 0,
    "input2": 1,
    "input3": 1,
    "input4": 1
  },
  "outputs": {
    "load1": 0,
    "load2": 0,
    "load3": 0,
    "load4": 0,
    "buzzer": 0
  },
  "modbus": {
    "cwt1": {
      "hum": 45.70000076,
      "temp": 31.89999962
    },
    "cwt2": {
      "hum": 44.29999924,
      "temp": 32.40000153
    }
  }
}
}
```

Hasil ini menunjukkan bahwa semua data sensor dan aktuator dapat terbaca dan berhasil publish melalui MQTT.

Selanjutnya untuk pengujian penambahan perangkat I2C dilakukan di Lab PTIO.



Gambar 4.4 Wiring Diagram Sensor INA219

Pengujian ini menggunakan satu unit ESP32 dengan firmware yang sama kemudian dihubungkan langsung ke modul **INA219** melalui jalur komunikasi I2C (pin SDA dan SCL) seperti pada Gambar 4.4. Konfigurasi dilakukan langsung melalui antarmuka web dengan memasukkan parameter nama sensor, tipe sensor, alamat I2C, serta alokasi pin SDA dan SCL.

SMART FARM A 1.0
AEROPONIK IoT EDGE SYSTEM

Status
Device
Wi-Fi
MQTT
GPIO
Modbus
I2C
Local Control
Firmware Update

I2C Sensors

Tambahkan sensor I2C (INA219, BME280, DHT12). Pastikan alamat dan pin SDA/SCL sesuai wiring perangkat.

power_monitor (INA219) @ 0x40 Edit Remove

Sensor Name	Sensor Type	I2C Address	SDA Pin
New I2C Sens	INA219	0x40	21

SCL Pin: 22 Done

+ Add I2C Sensor

Save & Reboot All I2C

Gambar 4.5 Konfigurasi Sensor I2C

Ketika tombol "Save & Reboot" ditekan pada antarmuka, server web internal ESP32 mengompilasi parameter formulir dan menyimpannya di config.json.

Payload JSON telemetry yang dipancarkan ESP32 ke topik MQTT smartfarm/8C94DF6BCDFC/telemetry secara otomatis bertambah dan memuat data dari sensor baru. Berikut data yang dikirimkan sebagai telemetry:

```
{
  "node_id": "8C94DF6BCDFC",
  ...
},
"telemetry": {
  "outputs": {},
  "i2c": {
    "power_monitor": {
      "bus_voltage_v": 1.011999965,
      "shunt_voltage_mv": 0.029999999,
      "current_ma": 0.100000001,
      "power_mw": 0
    }
  }
}
}
```

Secara keseluruhan pengguna cukup berinteraksi dengan antarmuka *Captive Web Portal* untuk melakukan konfigurasi tanpa menulis manual kode program.

4.2 Pengujian Alur Integrasi End-to-End via Antarmuka Dashboard

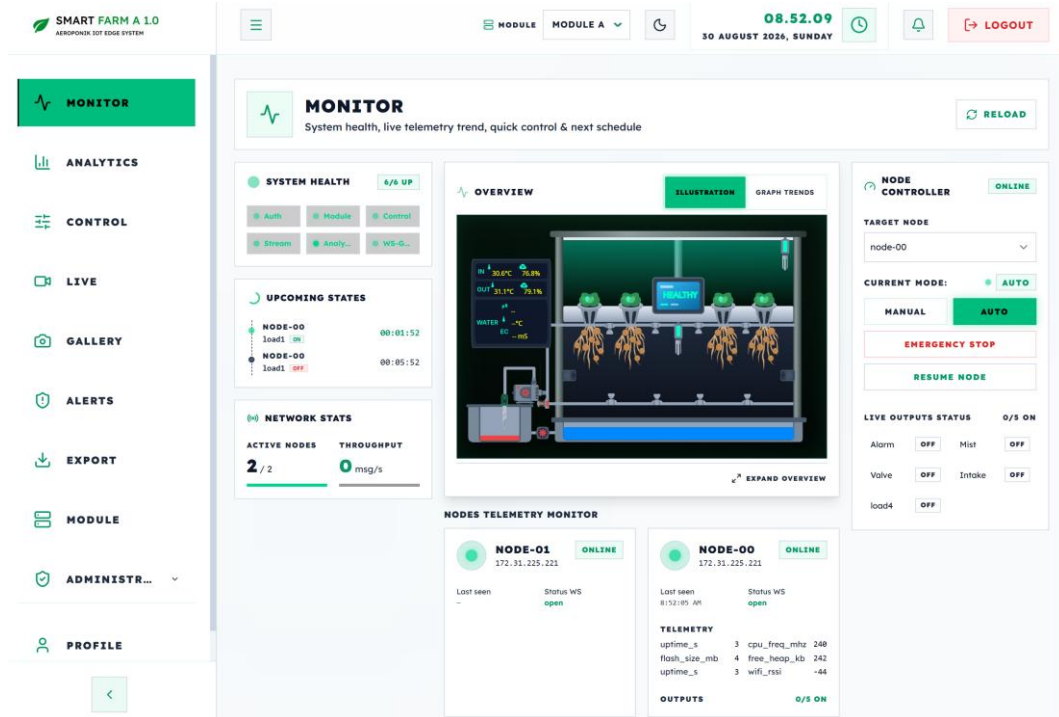
Setelah lapisan firmware terverifikasi mampu mengirimkan data secara dinamis, pengujian dilanjutkan ke alur operasional end-to-end yang dilakukan langsung melalui antarmuka *Dashboard*.

4.2.1 Registrasi Module, Node, dan Tag Mapping via Dashboard

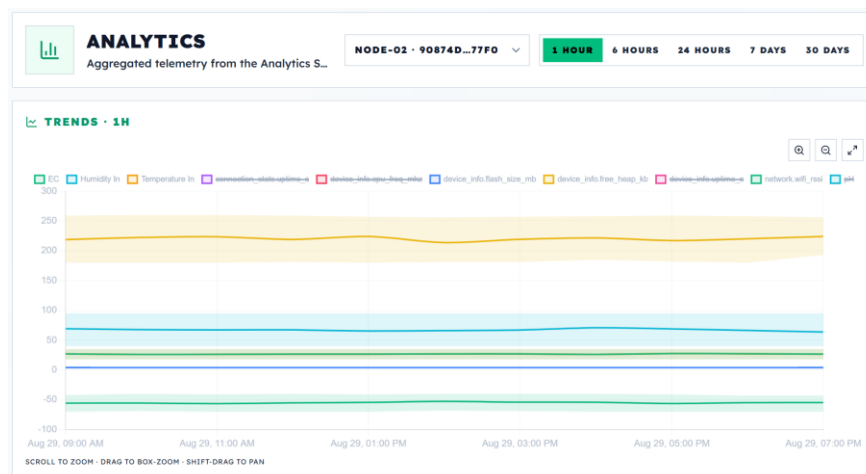
Untuk persiapan sebelum mendaftarkan perangkat, service wajib dijalankan dengan *docker compose up -d* untuk menjalankan semua service sekaligus. Kemudian *docker ps* untuk melihat apakah semua service sudah berjalan normal, jika sudah dashboard dapat diakses melalui *browser* dengan *url* localhost:5173. Pengguna mengarahkan browser ke halaman login dan memasukkan kredensial administrator default *username* = admin dan *password* = admin1234. Setelah berhasil masuk pengguna diharuskan untuk melakukan registrasi *module*, *node*, dan melakukan *tag-mapping*, untuk panduan konfigurasi lengkap terdapat pada Lampiran D.1.

4.2.2 Verifikasi Tampilan Dashboard, Analytics dan Pengujian Control

Pada halaman *Monitor*, data telemetry yang sudah di *mapping* langsung ditampilkan pada section overview begitu juga dengan kondisi aktuator. Data diterima melalui koneksi WebSocket persisten WS-Gateway (/ws) dengan latensi yang rendah.



Gambar 4.6 Tampilan Halaman Monitor

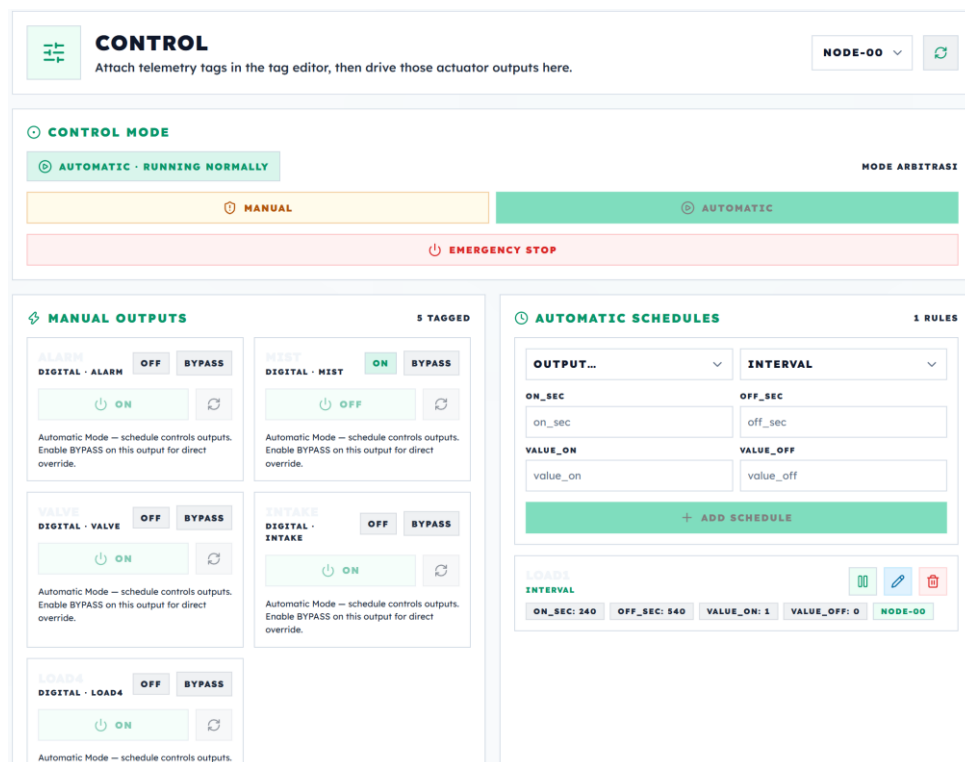


Gambar 4.7 Grafik Historis Telemetry



Gambar 4.8 Hasil Pengolahan Data Halaman Analytics

Data telemetri mentah tersimpan dalam *hypertable* TimescaleDB dan diolah secara agregat pada halaman *Analytics*, menghasilkan grafik historis fluktuasi iklim mikro.



Gambar 4.9 Tampilan Halaman Control

Melalui halaman *Control*, operator dapat menguji kontrol aktuator, secara default sistem akan menjalankan mode otomatis yang mengikuti penjadwalan. Kemudian mode dipindah manual untuk bisa mengaktifkan langsung.

Ketika tombol diaktifkan dashboard akan mengirimkan request POST `/v1/control/commands` via Kong Gateway. Kemudian Control Service meneruskan instruksi ke ESP32. Selanjutnya ESP32 akan mengaktifkan relai aktuator fisik dan mengirimkan sinyal konfirmasi.

COMMAND LOG

100 ENTRIES

OUTPUT	TYPE	VALUE	STATUS	TIME
LOAD1	INTERVAL	1	acked	1:40:47 PM
LOAD1	INTERVAL	0	acked	1:31:47 PM
LOAD1	INTERVAL	1	acked	1:27:45 PM
LOAD1	INTERVAL	0	acked	1:18:45 PM
LOAD1	INTERVAL	1	acked	1:14:44 PM
LOAD1	INTERVAL	0	acked	1:05:44 PM
LOAD1	INTERVAL	1	acked	1:01:43 PM
LOAD1	INTERVAL	0	acked	12:52:42 PM
LOAD1	INTERVAL	1	acked	12:48:40 PM
LOAD1	INTERVAL	0	acked	12:39:40 PM

PREV

1 / 10

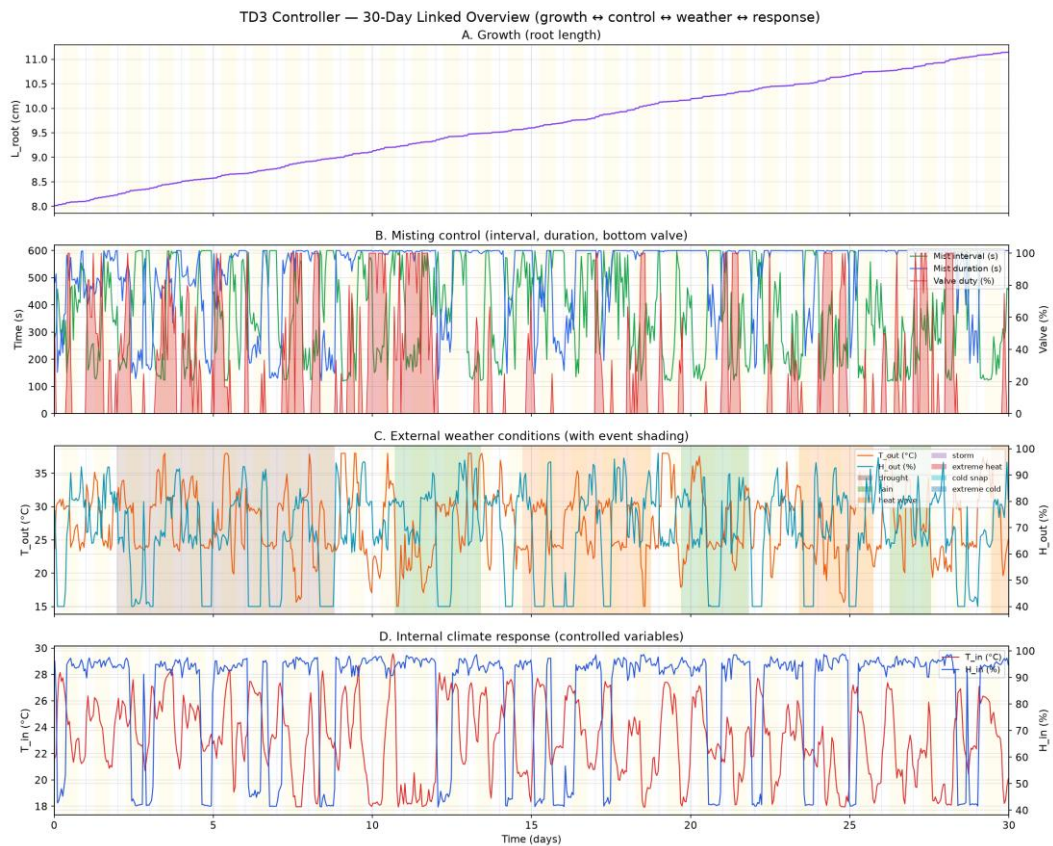
NEXT

Gambar 4.10 Tampilan Command Log Konfirmasi Aktuator

Control Service memvalidasi konfirmasi dan mengembalikan status sukses ke *dashboard* seperti pada Gambar 4.10. Indikator sakelar pompa pada *dashboard* berubah warna menjadi hijau tertanda aktif.

4.3 Hasil Pengujian Algoritma RL TD3

Pengujian model TD3 terlatih dievaluasi secara deterministik selama 30 hari kontinu (1 episode, 3.078 siklus, dt terakumulasi $\sim 2,59$ juta detik) pada lingkungan *AeroponicSimulatorEnv*. Simulator dijalankan dalam mode kontinu dengan regenerasi *event* cuaca acak seperti *heat*, *cold*, *drought*, *storm*, hingga *rain* guna menguji ketahanan agen secara acak. Hasil pengamatan divisualisasikan melalui Gambar 4.11 dalam dashboard 4-panel ber-sumbu-x sama, yang mencakup pertumbuhan akar (L_{root}), aksi kontrol *misting* (interval, durasi, *valve*), kondisi cuaca eksternal beserta *shading event*, serta respons iklim internal (T_{in} , H_{in}).



Gambar 4.11 30-Day Simulation Linked Overview (Growth, Control, Weather, Response)

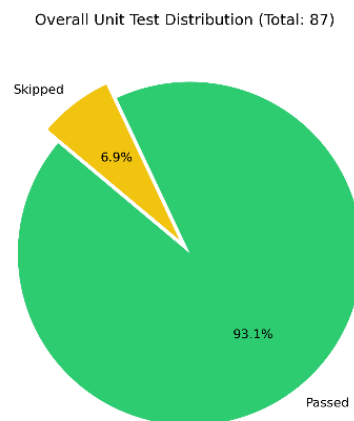
Hasil evaluasi menunjukkan pertumbuhan akar tanaman aeroponik berjalan sangat stabil dan naik mulus secara harian dari 8,02 cm menjadi 11,14 cm (+3,12 cm) tanpa adanya kematian prematur (*death cascade*). Agen TD3 mengeksekusi aksi kontrol adaptif dengan rata-rata interval *misting* ~345 detik, durasi *misting* ~502 detik, serta pengaktifan *bottom valve* sebesar 22,6% dari total siklus. Meskipun sistem dihantam ratusan siklus gangguan ekstrem termasuk *drought* (759 siklus), *extreme heat* (515 siklus), *rain* (299 siklus), *heat wave* (282 siklus), *extreme cold* (278 siklus), *cold snap* (207 siklus), dan *storm* (141 siklus) kontroler mampu merespons secara instan. Hal ini menjaga parameter iklim internal tetap optimal pada rentang T_{in} 17,9-29,7°C, dengan nilai akhir EC 1,54 dan pH 6,03, sekaligus membuktikan keandalan agen TD3 dalam mempertahankan kelangsungan hidup tanaman secara penuh. Untuk hasil lengkap model selama pelatihan dapat dilihat pada Lampiran M.1 dan hasil pengujian 30 hari pada Lampiran M.2

4.4 Evaluasi Tahap I: Konfigurasi Minimal tanpa Service Tambahan

Pengujian performa dan ketahanan sistem dilakukan secara bertahap. Tahap I menetapkan performa dasar sistem ketika hanya menjalankan layanan inti (Auth, Module, Analytics, Control, Stream, Alert, Notification, WS-Gateway, Kong, DBs, NATS, MQTT) dengan beban kerja 1 Modul dan 4 Node ESP32 yang diaktifkan.

4.4.1 Pengujian Unit Test Fitur Layanan

Sebagai persiapan awal dilakukan pengujian unit test untuk memastikan semua fitur berjalan normal. Pengujian ini dilakukan dengan menjalankan program pengujian secara otomatis.



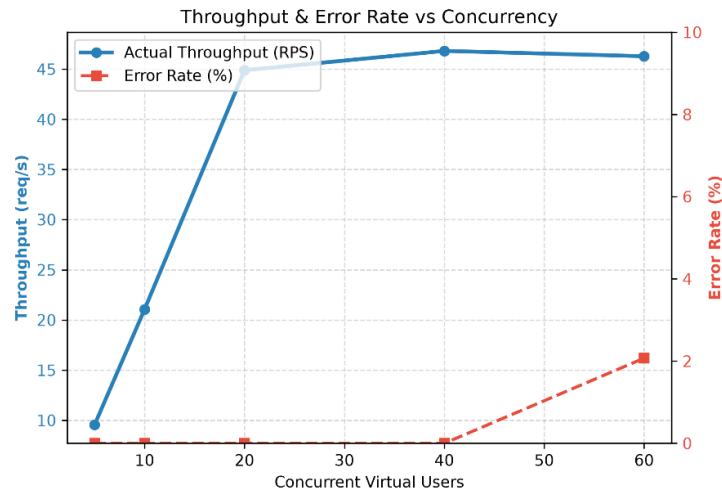
Gambar 4.12 Overall Unit Test Distribution Phase 1

Secara keseluruhan dari 87 fitur yang ada sebanyak 93.1% Passed dan 6.9% Skipped. Ini membuktikan bahwa keseluruhan fitur yang diuji berjalan normal. Nilai 6.9% Skipped bukanlah indikator gagal melainkan karena beberapa fitur yang memerlukan data uji belum bisa didapatkan kemudian unit test tersebut diabaikan agar tidak mengganggu pengetesan selanjutnya.

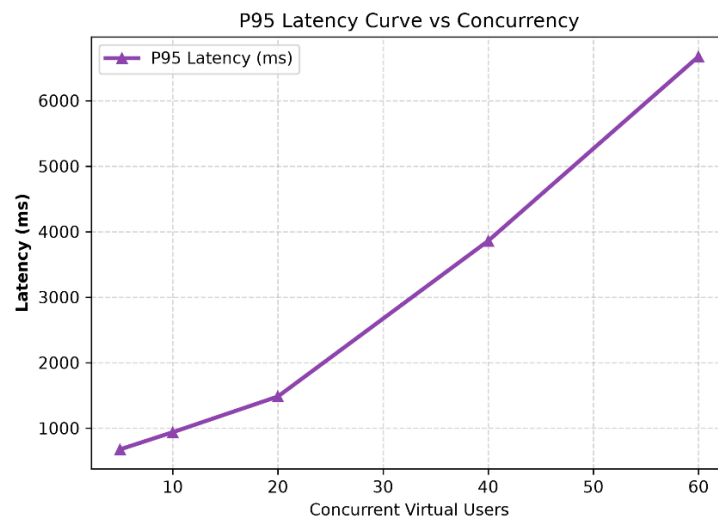
4.4.2 Pengujian Stress Test

Pengujian stress test Tahap I dibagi menjadi dua bagian untuk membuktikan Skalabilitas Selektif dari dua sisi: (1) beban API pada gateway REST (Kong), dan (2) beban ingesti telemetry ujung-ke-ujung (MQTT → NATS → TimescaleDB).

Untuk uji beban API pengujian dilakukan pada layanan Kong API Gateway (/v1) terhadap 10 Layanan Inti. Pengujian ini dilakukan untuk mendapatkan **Breakpoint Capacity Test (Pencarian Batas Throughput)**: 5 level konkurensi (5, 10, 20, 40, 60 *users*) dengan target 10–500 RPS, masing-masing 8 detik.



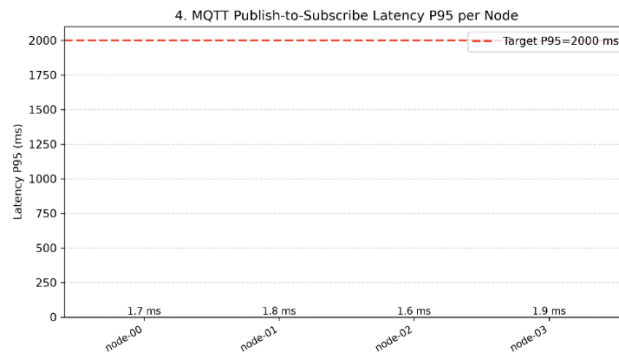
Gambar 4.13 Throughput & Error vs Concurrency Phase 1



Gambar 4.14 P95 Latency & Error vs Concurrency Phase 1

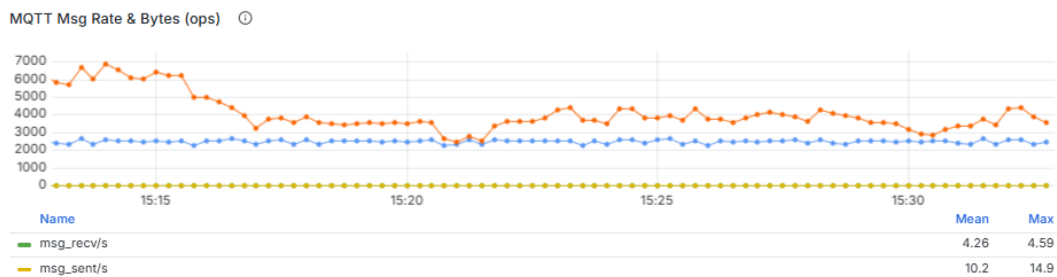
Hasil pengujian menunjukkan Gambar 4.13 *throughput* terhadap konkurensi naik cepat hingga mencapai puncak **51 RPS** pada Level 3 (20 *users*) dengan Latensi P95 pada Gambar 4.14 sebesar **1.482 ms**. Sedangkan error baru muncul setelah 60 user sebesar di **2,1%**.

Uji beban di sisi ingesti telemetry diuji dengan beban 4 node dalam 1 module. Artinya dalam satu module terdapat 4 node yang masing-masing node membaca beberapa sensor. Pengujian ini untuk menguji Latensi P95 untuk publish-subscribe, publish throughput (msg/s Mosquitto); ingest throughput (in_msg/s NATS), *transaction* TimescaleDB.



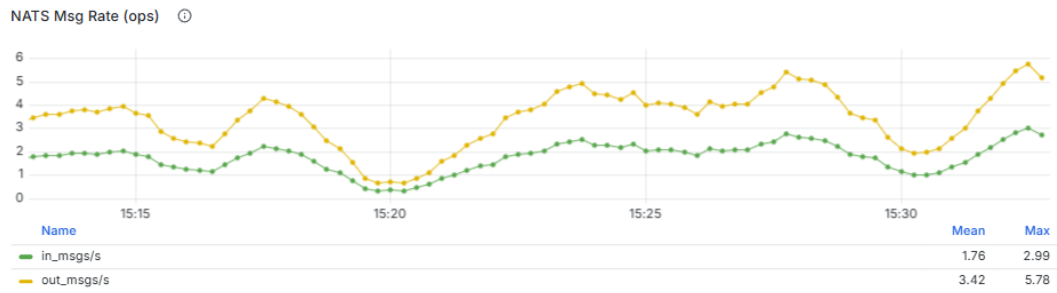
Gambar 4.15 MQTT Pub-Sub Latency P95 Phase 1

Gambar 4.15 menunjukkan hasil pengujian latensi transmisi *publish-subscribe* yang dilakukan secara simultan menggunakan **empat node**, diperoleh rata-rata latensi P95 yang sangat rendah, yaitu **1.75 ms** dengan batas toleransi maksimum yaitu **2000 ms** (2 detik).



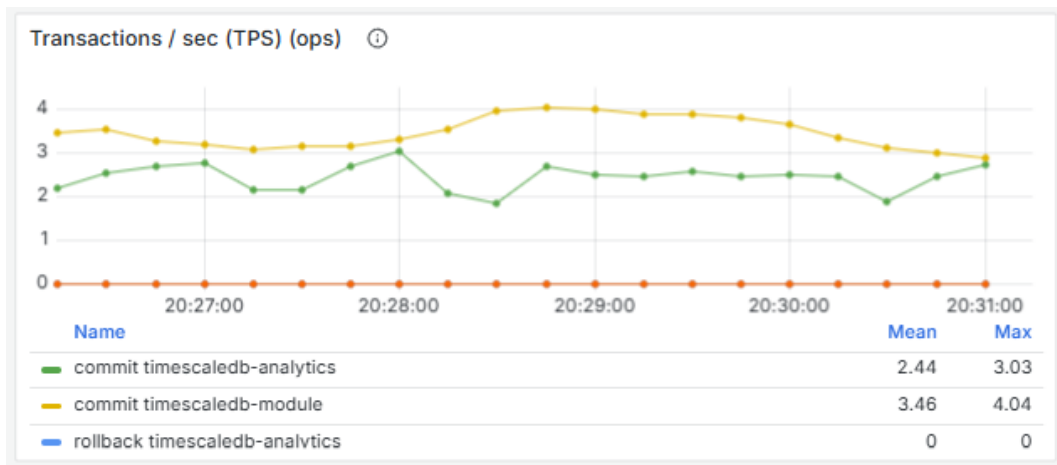
Gambar 4.16 Publish Troughput MQTT Phase 1

Gambar 4.16 menunjukkan hasil pengujian troughput MQTT pada broker menunjukkan bahwa rata – rata troughput yang diterima broker adalah **4.26 msg/s**. Ini sesuai dengan konfigurasi node yang mengirimkan telemetry setiap 1 detik sekali.



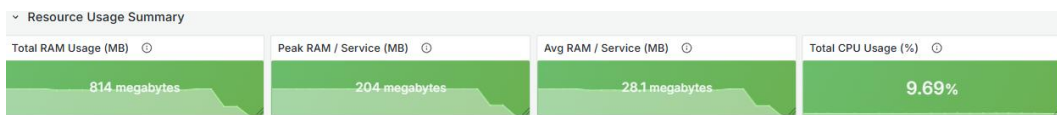
Gambar 4.17 Ingest Throughput (in_msg/s NATS) Phase 1

Gambar 4.17 menunjukkan hasil pengujian rata-rata laju ingesti data stabil berada pada angka **1,76 msg/s** dengan beban puncak maksimum mencapai **2,99 msg/s**.



Gambar 4.18 Transaction in TimescaleDB (tps) Phase 1

Gambar 4.18 mengilustrasikan rata-rata laju transaksi komit pada basis data TimescaleDB oleh *Module Service* dalam jendela waktu 5 menit terakhir, dengan nilai rata-rata sebesar **3,46 tps** dan batas puncak maksimum **4,04 tps**. Untuk Analytics Service nilai rata-rata **2.44 tps** dan batas puncak maksimum **3.03 tps**.



Gambar 4.19 Resource Usage Summary Phase 1

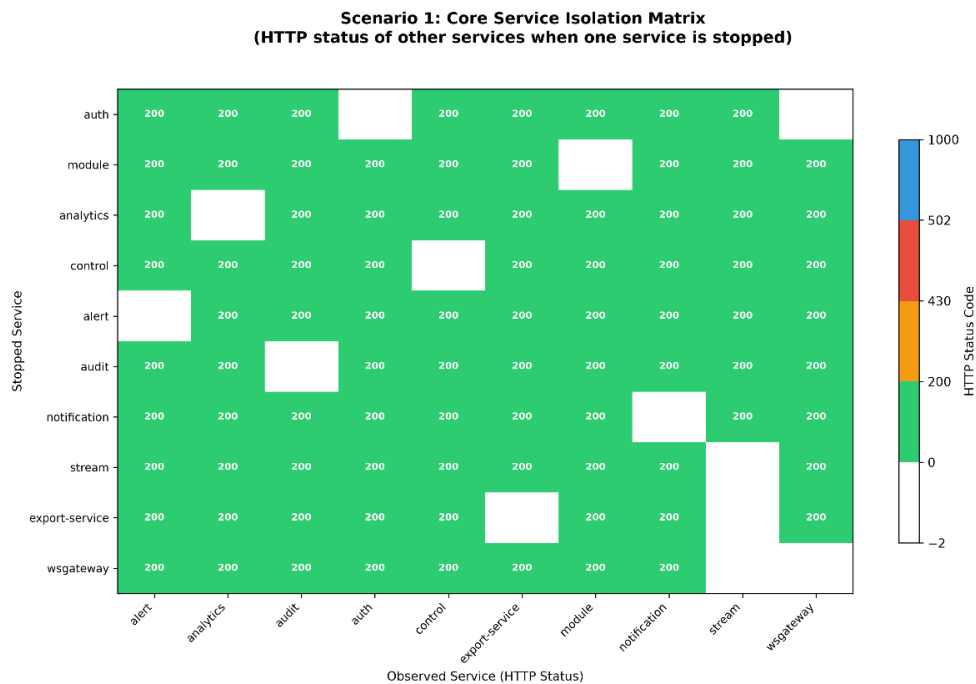
Selama fase pengujian beban, utilisasi sumber daya pada server *backend* menunjukkan efisiensi yang sangat tinggi seperti yang ditunjukkan pada Gambar

4.19 dengan beban CPU puncak hanya mencapai **9.69%** dan total konsumsi RAM keseluruhan sebesar **0.8 GB**. Di tingkat kontainer, rata-rata konsumsi memori per layanan mikro (*microservice*) berada pada angka yang sangat ringan, yaitu **28.1 MB**, dengan batas konsumsi puncak maksimum per layanan paling intensif terbatas sebesar **200 MB**.

4.4.3 Pengujian Chaos Engineering

Pengujian ini dilakukan untuk menganalisis Isolasi Kegagalan pada 10 Core Microservices yang berjalan tanpa AI/ML. Lima skenario chaos dirancang untuk mencoba memutuskan rantai komunikasi antar *bounded context* dan mengamati apakah kegagalan satu layanan merembet ke layanan lain (*cascade failure*):

1. Core Service Isolation Matrix: mematikan setiap layanan inti satu per satu dan memverifikasi bahwa layanan lain tetap beroperasi (HTTP 200). Menguji prinsip *failure domain isolation*.
2. Database & Cache Degradation Isolation: mematikan database individual (mariadb-module, mariadb-alert, mariadb-audit, mariadb-stream, mariadb-notification, redis-shared) untuk memastikan kegagalan database tidak menyeret service lain.
3. Kong Partial Backend Failure: mematikan beberapa layanan auxiliary sekaligus (alert, stream, audit, export-service) dan memverifikasi Kong tetap melayani healthy services.
4. Event Bus Blackhole & Reconnection: mematikan NATS broker selama 30 detik, memverifikasi REST API tetap berfungsi, dan memastikan auto-reconnection setelah recovery.
5. Cascading Failure Prevention: mematikan module service (paling terhubung) dan memverifikasi Auth, Analytics, Control, dan Alert tidak mengalami *cascade failure* (tidak return 500).



Gambar 4.20 Core Service Isolation Matrix Phase 1

Gambar 4.20 merupakan hasil pengujian pada skenario pertama menunjukkan 100% Lulus. Tidak ada respon HTTP yang gagal pada service lain saat suatu service tertentu dihentikan. Selain pengujian pada skenario pertama, hasil pengujian lainnya juga menunjukkan keberhasilan dengan tidak adanya kegagalan dalam setiap skenario. Untuk hasil lengkap pengujian skenario lain dapat dilihat pada Lampiran L.3.

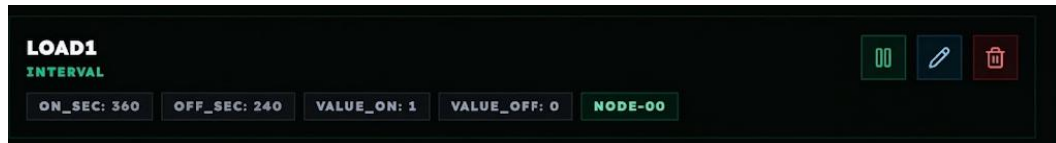
4.5 Evaluasi Tahap II: Konfigurasi Lengkap dengan Service Tambahan

Pada **Tahap II**, sistem dievaluasi pada kapasitas operasional penuh. Pengujian ini mengintegrasikan seluruh komponen tambahan Seperti ML, model-control, dan model- controller. Untuk detail konfigurasi dapat dilihat di lampiran.

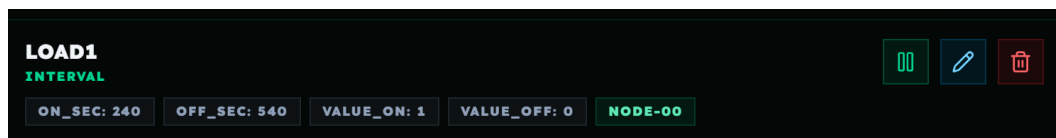
4.5.1 Integrasi Layanan Tambahan ML, model-control, model-controller

Pengujian ini membuktikan bahwa komponen kontrol adaptif bekerja secara *end-to-end*: dari deteksi visual, inferensi, penggabungan data, penentuan keputusan RL, hingga pembaruan jadwal otomatis ke Control Service.

Sebelum memulai pengujian layanan tambahan harus dikonfigurasi terlebih dahulu untuk detail konfigurasi layanan tambahan terdapat pada Lampiran K.1 untuk konfigurasi *Environment Variable* dan Lampiran K.2 untuk setup service.



Gambar 4.21 Kondisi Penjadwalan Waktu Siang



Gambar 4.22 Kondisi Penjadwalan Waktu Malam

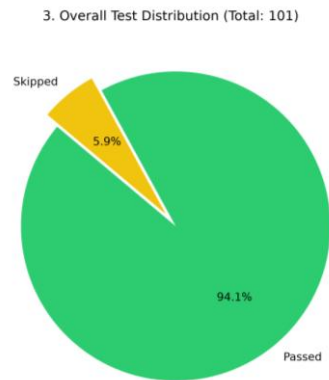
Pada pengujian model-control menampilkan log kontrol adaptif sebagai berikut:

```
model-control-1 | cache metrics: {"T_in": {"value": 36.60000229, "age_s": 15.5},
"H_in": {"value": 36.79999924, "age_s": 2.7}, "T_out": {"value": 32.20000076,
"age_s": 3.8}, "H_out": {"value": 45.5, "age_s": 4.5}, "EC": {"value": 1.5, "age_s":
Infinity}, "pH": {"value": 6.5, "age_s": Infinity}, "T_nut": {"value": 25.0,
"age_s": Infinity}}
model-control-1 | state=[46.2, 0.25, 30.0, 36.79999924, 30.0, 45.5, 1.5, 6.5, 25.0,
0.0]
model-control-1 | INFO:httpx:HTTP Request: POST http://model-
controller:8080/predict "HTTP/1.1 200 OK"
model-control-1 | tick skipped cycle_done=False pending=True elapsed=383.7s
cycle=780
```

Pengujian ini membuktikan kontrol adaptif telah berhasil bekerja yang dibuktikan dengan berubahnya durasi *misting* saat siang dan malam yang dapat dilihat pada Gambar 4.21 dan 4.22. Selain itu juga metadata visual dari MinIO berhasil digabung dengan parameter sensor, diteruskan ke algoritma RL.

4.5.2 Pengujian Unit Test Fitur Layanan

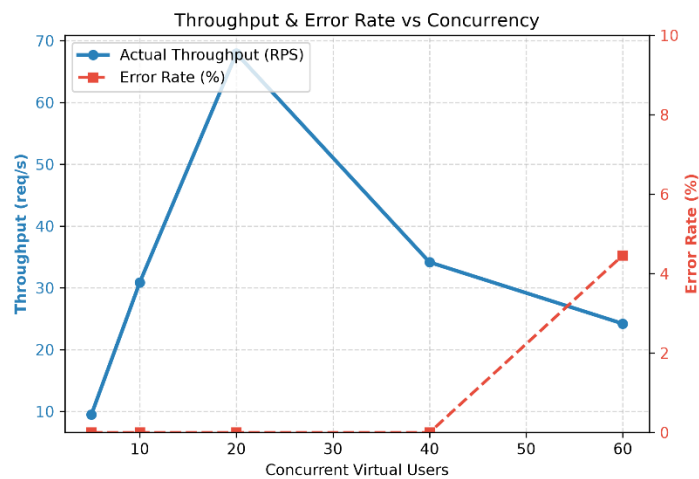
Pengujian unit test mencakup seluruh 13 microservices beserta modul AI/ML dan Stream Service dengan total **101** fitur yang diuji:



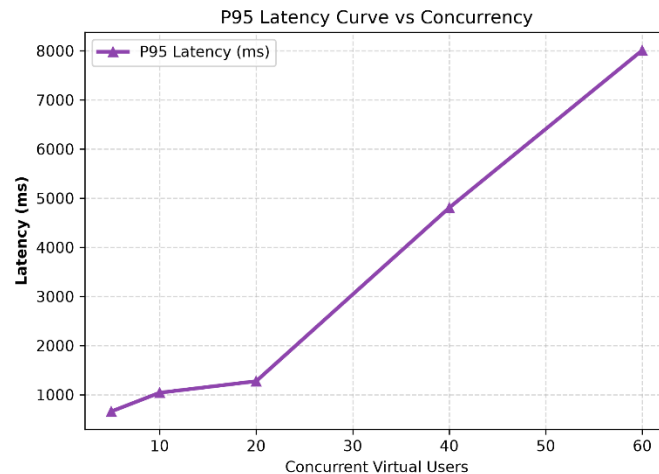
Gambar 4.23 Overall Test Distribution Fase 2

Secara keseluruhan semua fitur berjalan normal dan lulus dan siap digunakan untuk pengujian selanjutnya.

4.5.3 Pengujian Stress Test



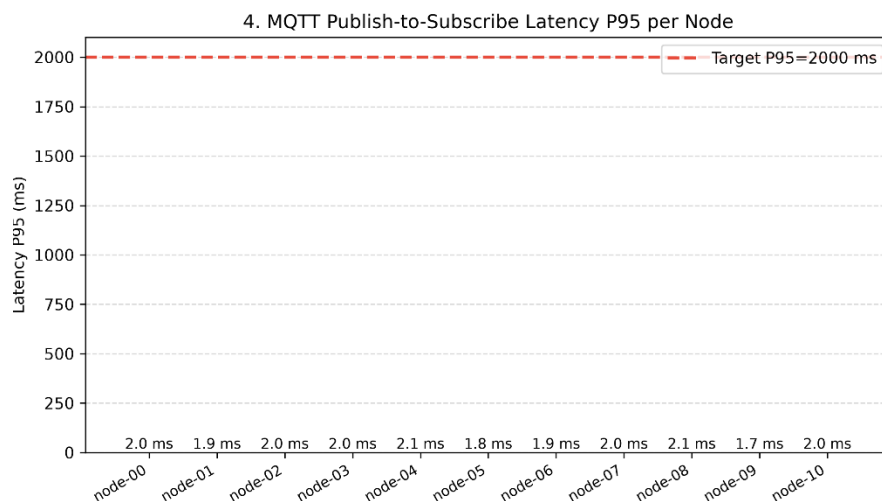
Gambar 4.24 Throughput & Error vs Concurrency Phase 2



Gambar 4.25 Latency vs Concurrency Phase 2

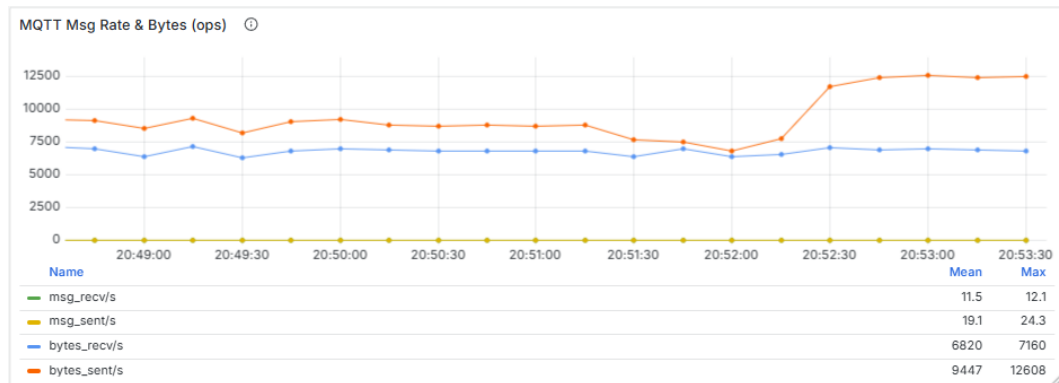
Gambar 4.24 dan 4.25 menunjukkan grafik *throughput* terhadap konkurensi naik cepat hingga mencapai puncak **68 RPS** pada Level 3 (20 *users*) dengan Latensi P95 **1275 ms**. Kemudian turun drastis **34.2 RPS** Sedangkan error baru muncul setelah 60 user sebesar di **4,5%**.

Uji beban di sisi ingesti telemetry diuji dengan beban 10 node dalam 5 module sehingga setiap module memiliki dua node.



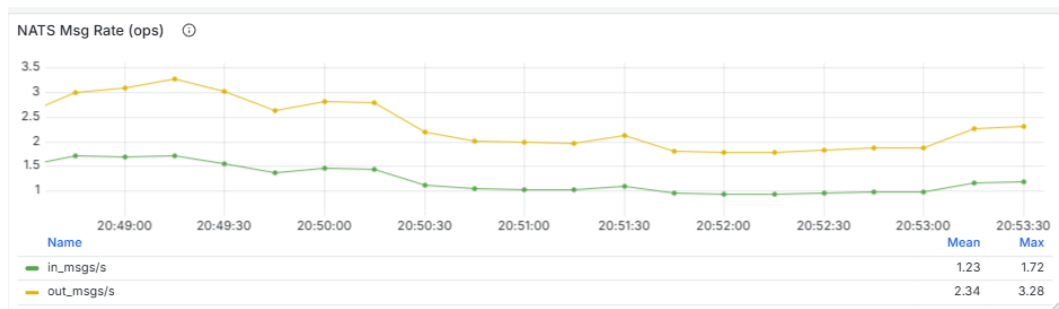
Gambar 4.26 MQTT Pub-Sub Latency P95 Phase 2

Berdasarkan hasil pengujian latensi transmisi *publish-subscribe* yang dilakukan secara simultan menggunakan **empat node**, diperoleh rata-rata latensi P95 yang sangat rendah, yaitu **2.1 ms** dengan batas toleransi maksimum yaitu **2000 ms** (2 detik).



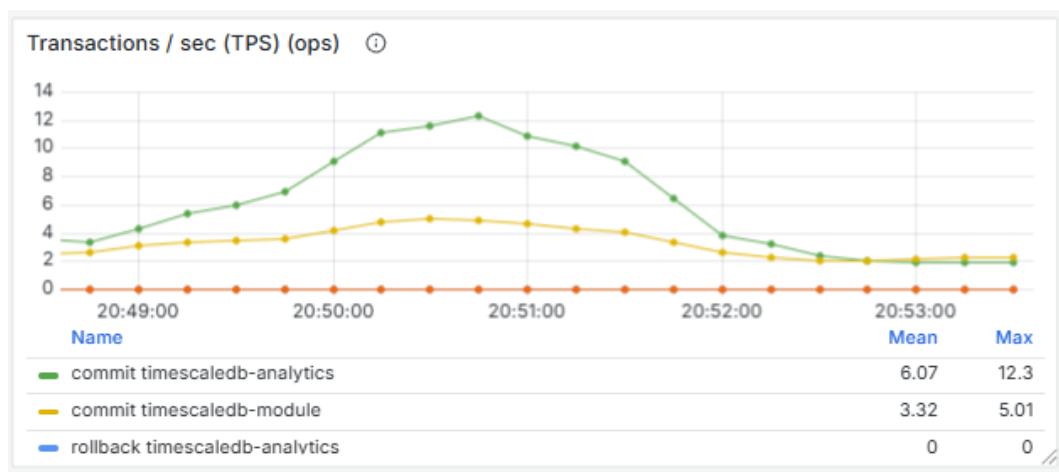
Gambar 4.27 Publish Troughput MQTT Phase 2

Hasil pengujian troughput MQTT pada broker menunjukkan bahwa rata – rata troughput yang diterima broker adalah **11.5 msg/s**. Ini sesuai dengan konfigurasi setiap node yang mengirimkan telemetri setiap 1 detik sekali.



Gambar 4.28 Ingest Throughput (in_msg/s NATS) Phase 2

Berdasarkan grafik pengujian tersebut, rata-rata laju ingesti data stabil berada pada angka **1,2 msg/s** dengan beban puncak maksimum mencapai **1,72 msg/s**.



Gambar 4.29 Transaction in TimescaleDB (tps) Phase 2

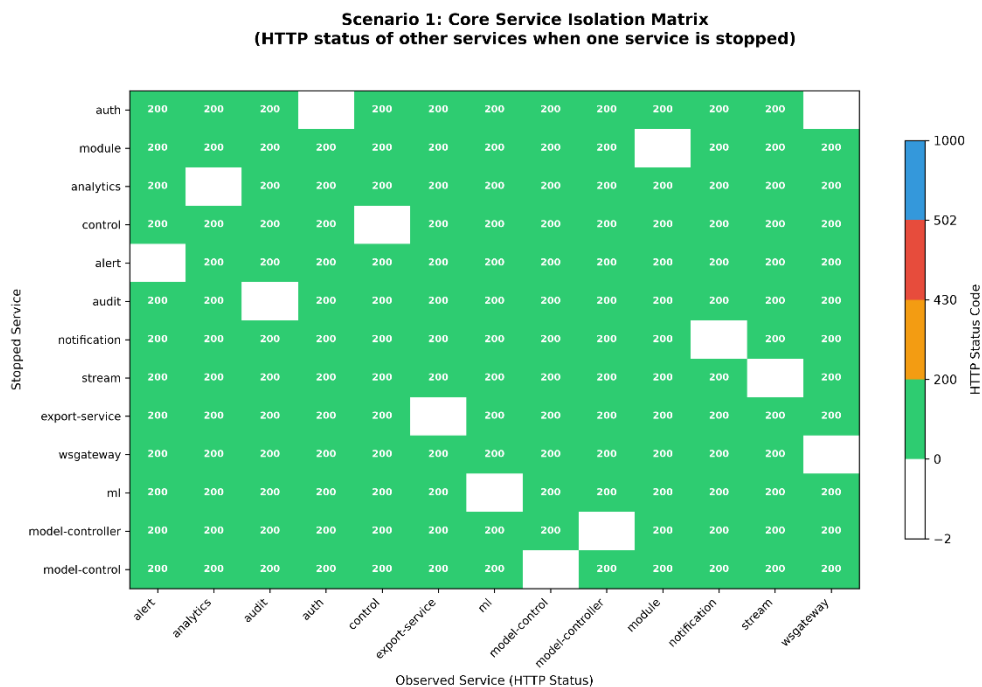
Rata-rata laju transaksi komit pada basis data TimescaleDB oleh *Module Service* dalam jendela waktu 5 menit terakhir, dengan nilai rata-rata sebesar **3,32 tps** dan batas puncak maksimum **5,01 tps**. Untuk Analytics Service nilai rata-rata **6.07 tps** dan batas puncak maksimum **12.3 tps**.



Gambar 4.30 Resource Usage Summary Phase 2

Selama fase pengujian beban, utilisasi sumber daya pada server *backend* menunjukkan efisiensi yang sangat tinggi dengan beban CPU puncak mencapai **27.5%** dan total konsumsi RAM keseluruhan sebesar **1.7 GB**. Di tingkat kontainer, rata-rata konsumsi memori per layanan mikro (*microservice*) berada pada angka yang sangat ringan, yaitu **54.6 MB**, dengan batas konsumsi puncak maksimum per layanan paling intensif terbatas sebesar **523 MB**.

4.5.4 Pengujian Chaos Engineering Test



Gambar 4.31 Core Service Isolation Matrix Phase 2

Untuk skenario pengujian ini sama dengan fase 1 namun untuk jumlah service yang diuji bertambah menjadi 13 service. Namun hasil pengujian tetap menunjukkan ketahanan sistem yang sangat baik seperti pengujian pada skenario 1 Gambar 4.31. Seluruh skenario *chaos engineering* berhasil dilalui dengan kelulusan 100%. Untuk hasil lengkap pengujian skenario lain dapat dilihat pada Lampiran L.6.

4.6 Analisis Komparatif dan Pembahasan

Bagian ini membandingkan data empiris dari tahap I dan tahap II untuk menganalisis dampak modularitas arsitektur terhadap performa dan keandalan sistem.

4.6.1 Analisis Kinerja Troughput Latensi dan Resource

Tabel 4.1 Ringkasan Kinerja Troughput Latensi dan Resource

Parameter Pengujian	Tahap I	Tahap II
Beban Ideal User	20 User	20 User
Throughput Puncak	51.3 RPS	68 RPS
Latensi P95 API	1482 ms	1275 ms.
Tingkat Kesalahan (Error Rate)	2,1%.	4,5%.
Latensi MQTT	1.75 ms	2.1 ms
Troughput MQTT	4.26 msg/s	11.5 msg/s
Laju Ingesti NATS	1,76 - 2,99 msg/s	1,2 - 1,72 msg/s
TimescaleDB Transaction	3,46 - 4,04 tps (module) 2.44 - 3.03 tps (analytics)	3.32 - 5.01 tps (module) 6.07 - 12.3 tps (analytics)
Total RAM	800 MB	1700 MB
Average RAM/Service	28.1 MB	54.6 MB
Peak RAM/Service	200 MB	523 MB
Peak CPU	9.69%	27.5%

Berdasarkan Tabel 4.1 menunjukkan bahwa performa REST API Gateway mencapai titik ideal pada beban **20 users** dengan peningkatan throughput puncak dari **51,3 RPS** menjadi **68 RPS** serta penurunan latensi P95 dari **1482 ms** ke **1275 ms**, meskipun diiringi kenaikan error rate tipis dari **2,1%** menjadi **4,5%**. Di sisi ingesti telemetry, sistem terbukti sangat responsif dan andal dengan latensi MQTT yang stabil di kisaran **1,75–2,1 ms**, peningkatan throughput MQTT dari **4,26 msg/s** ke **11,5 msg/s**, serta lonjakan pemrosesan transaksi TimescaleDB Analytics hingga **12,3 tps**. Kenaikan skala pemrosesan ini diimbangi oleh utilisasi sumber daya server yang sangat efisien, di mana konsumsi RAM total meningkat dari **800 MB** ke **1700 MB** dan peak CPU berada pada level aman **27,5%**.

4.6.2 Pengaruh Penambahan Node

Pelipatgandaan perangkat fisik dari 4 node (1 modul) menjadi 10 node (5 modul) meningkatkan volume pesan telemetry secara signifikan tanpa menimbulkan *bottleneck* pada sistem. Berikut adalah ringkasan perubahan kinerja dari Tahap I ke Tahap II:

1. **Peningkatan *Throughput* MQTT:** Naik drastis dari **4,26 msg/s** menjadi **11,5 msg/s** (+170%), mencerminkan efisiensi transmisi data *edge* saat beban perangkat bertambah.
2. **Lonjakan Transaksi Analitik TimescaleDB:** Laju transaksi *Analytics Service* melonjak dari **2,44–3,03 tps** menjadi **6,07–12,3 tps** (naik 2x hingga 4x lipat), mengindikasikan fitur *hypertable partitioning* bekerja lebih efisien pada volume data tinggi.
3. **Stabilitas Latensi MQTT:** Latensi transmisi naik tipis dari **1,75 ms** menjadi **2,1 ms** (+0,35 ms / +20%), namun tetap sangat cepat dan memenuhi ambang batas toleransi.
4. **Penyesuaian *Ingest Rate* NATS:** Laju ingesti NATS berubah dari **1,76–2,99 msg/s** menjadi **1,2–1,72 msg/s** akibat perubahan pola *batching* internal broker.

5. **Stabilitas Transaksi Modul TimescaleDB:** Transaksi *Module Service* relatif stabil pada rentang **3,32–5,01 tps**.

4.6.3 Pengaruh Penambahan Layanan AI/ML

Berikut ini adalah beberapa pengaruh penambahan Layanan AI/ML:

1. **Melonjaknya *Throughput*:** Kapasitas puncak naik dari **51,3 RPS menjadi 68 RPS** (+32,5%) meskipun layanan AI/ML tidak menerima beban kueri langsung.
2. **Membaiknya Latensi:** Latensi P95 turun dari **1482 ms ke 1275 ms** (lebih cepat 207 ms / -14%).
3. **Kenaikan *Error Rate*:** *Error rate* naik dari **2,1% menjadi 4,5%**, namun masih berada pada batas aman (<10%).
4. **Lonjakan *Resource Usage*:** Konsumsi RAM melonjak dari **800 MB ke 1700 MB** (+112,5%) dan *Peak CPU* naik dari **9,69% ke 27,5%** (+184%).
5. **Mekanisme Kontrol:** Kontrol adaptif berhasil memperbarui penjadwalan yang sudah berjalan pada Control Service.

4.6.4 Keandalan Sistem Menangani Service yang Gagal

Berdasarkan hasil 5 skenario chaos engineering pada Tahap I dan II, kesimpulan keseluruhan sistem dalam menangani service yang gagal adalah:

1. Isolasi kegagalan berhasil karena tidak terjadi *cascade failure* selama 5 skenario dijalankan. Ketika satu layanan dihentikan baik core service, database, broker, gateway, maupun layanan AI/ML layanan lain tetap berjalan dan merespon dengan benar.
2. Semua layanan yang dihentikan pulih secara otomatis tanpa intervensi manual.
3. Arsitektur yang memungkinkan isolasi tiga mekanisme utama bekerja bersama:

- a. Database-per-Service: Kegagalan database satu layanan tidak memengaruhi layanan lain.
 - b. NATS JetStream (async): Pesan tetap tersimpan saat subscriber gagal, dilanjutkan setelah recovery.
 - c. Kong Gateway: Memfilter request ke service yang sehat, mencegah *thundering herd*.
4. Skalabilitas sistem teruji dan mampu berjalan ketika penambahan layanan (dari 10 menjadi 13 services) dan perangkat fisik (dari 4 menjadi 10 node) tanpa mengorbankan stabilitas. Dibuktikan semua pengujian Tahap I dan II mencapai 100% lulus.

BAB V

KESIMPULAN

4.7 Kesimpulan

Dari serangkaian proses perancangan, implementasi, hingga validasi sistem pemantauan dan kontrol lingkungan aeroponik berbasis arsitektur microservice, ditarik kesimpulan sebagai berikut:

1. Arsitektur Dual-Layer Modularity berhasil memisahkan dependensi perangkat keras di *edge* (ESP32) dan komputasi di *server* (13 *microservices*), sehingga kedua lapisan dapat dikembangkan dan diskalakan secara independen.
2. Penerapan *Factory Pattern* dan *Protocol Registry* pada ESP32 memungkinkan integrasi sensor/aktuator baru (seperti sensor I2C INA219) secara dinamis via *Captive Web Portal* / *config.json* tanpa mengubah kode program inti atau kompilasi ulang
3. Alur telemetri dan kontrol terintegrasi aman melalui MQTT (*edge*), NATS JetStream (*event bus*), Kong API Gateway, serta prinsip *database-per-service* yang terbukti mencegah kegagalan beruntun (*zero cascade failure*)
4. Model visi komputer (YOLOv8) dan *Reinforcement Learning* (TD3) berjalan sebagai *microservice* independen, memungkinkan pembaruan model kecerdasan buatan tanpa mengganggu layanan *core* operasional
5. Keandalan & kinerja. Chaos engineering membuktikan isolasi kegagalan sangat andal: Fase 1 = 43 PASS (0 failed) dan Fase 2 = 50 PASS (0 failed) (isolasi service, degradasi DB, *outage* NATS, *stop/recovery* Kong, *cascading* REST tetap 200, degradasi *graceful*, pulih otomatis). Pada *stress test* (beban penuh), latensi end-to-end rendah, throughput stabil, dan utilisasi terisolasi per service arsitektur tahan kegagalan parsial di konfigurasi minimal maupun beban penuh.

4.8 Saran

Berdasarkan temuan penelitian dan keterbatasan yang teridentifikasi, berikut adalah saran untuk pengembangan sistem lebih lanjut:

1. **Pelengkapan Protokol Sensor dan Aktuator pada Firmware:** Firmware ESP32 saat ini sudah mendukung GPIO Digital/Analog, Modbus RTU, dan I2C. Disarankan untuk melengkapi dukungan protokol lain yang umum digunakan di pertanian presisi, seperti 1-Wire (DS18B20), CAN bus, dan PWM/ADC khusus untuk aktuator, agar sistem dapat mengintegrasikan beragam jenis sensor dan aktuator tanpa perlu modifikasi arsitektur inti.
2. **Konfigurasi Terpadu dalam Satu Dashboard:** Saat ini konfigurasi firmware dilakukan melalui Captive Web Portal terpisah. Disarankan untuk mengintegrasikan konfigurasi firmware seperti penambahan sensor dan pin mapping langsung ke dalam dashboard utama, sehingga pengguna dapat mengelola seluruh sistem (node, sensor, aktuator, dan jadwal) dari satu antarmuka tanpa perlu beralih ke portal perangkat.
3. **Sinkronisasi Otomatis Aturan Kontrol Lokal (Edge Fallback):** Untuk mengantisipasi kondisi ketika peladen (*server*) atau jaringan terputus, disarankan agar *firmware* ESP32 dilengkapi dengan mekanisme kloning (*sync*) aturan konfigurasi jadwal/ambang batas (*schedule/threshold*) terakhir dari *server* ke dalam memori lokal (*LittleFS*)^[1]. Dengan demikian, perangkat *edge* dapat beralih secara otomatis ke mode *fail-safe* dan tetap menjalankan eksekusi kontrol lokal secara mandiri (*autonomous local control*) sesuai konfigurasi terakhir hingga koneksi ke peladen pulih kembali.

DAFTAR PUSTAKA

- [1] I. A. Lakhari, G. Jianmin, T. N. Syed, F. A. Chandio, N. A. Buttar, dan W. A. Qureshi, "Monitoring and Control Systems in Agriculture Using Intelligent Sensor Techniques: A Review of the Aeroponic System," *J. Sens.*, vol. 2018, no. 1, hlm. 8672769, 2018, doi: 10.1155/2018/8672769.
- [2] A. Lercher, J. Glock, C. Macho, dan M. Pinzger, "Microservice API Evolution in Practice: A Study on Strategies and Challenges," *J. Syst. Softw.*, vol. 215, hlm. 112110, Sep 2024, doi: 10.1016/j.jss.2024.112110.
- [3] E. Gamma, R. Helm, R. Johnson, dan J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Pearson Education, 1994.
- [4] L. Giamattei dkk., "Monitoring tools for DevOps and microservices: A systematic grey literature review," *J. Syst. Softw.*, vol. 208, hlm. 111906, Feb 2024, doi: 10.1016/j.jss.2023.111906.
- [5] C. Saldaña Enderica, J. R. Llata, dan C. Torre-Ferrero, "Guided Reinforcement Learning with Twin Delayed Deep Deterministic Policy Gradient for a Rotary Flexible-Link System," *Robotics*, vol. 14, no. 6, hlm. 76, Jun 2025, doi: 10.3390/robotics14060076.
- [6] A. Basiri dkk., "Chaos Engineering," *IEEE Softw.*, vol. 33, no. 3, hlm. 35–41, Mei 2016, doi: 10.1109/MS.2016.60.
- [7] H. Knoche dan W. Hasselbring, "Using Microservices for Legacy Software Modernization," *IEEE Softw.*, vol. 35, no. 3, hlm. 44–49, Mei 2018, doi: 10.1109/MS.2018.2141035.
- [8] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, dan M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," *IEEE Commun. Surv. Tutor.*, vol. 17, no. 4, hlm. 2347–2376, 2015, doi: 10.1109/COMST.2015.2444095.
- [9] D. C. Ma'soem dan N. Rukhviyanti, "Implementation of Event-Driven Architecture using NATS JetStream for a Large-Scale Online Exam Answer Collection System," *J. Bus. Soc. Technol.*, vol. 7, no. 2, hlm. 482–495, Mei 2026, doi: 10.59261/jbt.v7i2.654.
- [10] S. D. Kalamaras, M.-A. Tsitsimpikou, C. A. Tzenos, A. A. Lithourgidis, D. S. Pitsikoglou, dan T. A. Kotsopoulos, "A Low-Cost IoT System Based on the ESP32 Microcontroller for Efficient Monitoring of a Pilot Anaerobic Biogas Reactor," *Appl. Sci.*, vol. 15, no. 1, hlm. 34, Jan 2025, doi: 10.3390/app15010034.
- [11] J. Terven, D.-M. Córdova-Esparza, dan J.-A. Romero-González, "A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS," *Mach. Learn. Knowl. Extr.*, vol. 5, no. 4, hlm. 1680–1716, Des 2023, doi: 10.3390/make5040083.

LAMPIRAN



QR-Code Lampiran Lengkap



QR-Code Repository Github



QR-Code Dokumentasi Lainnya